Vision Computing Exam Prep Guide

PART 1 — COMPREHENSIVE LECTURE NOTES

---

1. IMAGES AND FILTERING

1.1 Digital Image Fundamentals

A digital image is a **discrete sampling** of a continuous scene. For a grayscale image, we have a function `I(x, y)` where `x ∈ [0, W−1]`, `y ∈ [0, H−1]`, and `I` is the intensity. For color, we have a vector `I(x, y) = [R, G, B]`.

Spatial resolution: Number of pixels (e.g., 1920×1080). Higher = more detail. **Intensity resolution**: Number of distinct gray levels (e.g., 8-bit = 256 levels). More bits = smoother gradients. **Sampling theorem**: To avoid aliasing, sample at more than twice the highest frequency (Nyquist rate). If you undersample, high frequencies fold into low frequencies (aliasing/moire patterns).

Image as a matrix: All image processing operations are matrix operations. Addition, subtraction, multiplication, convolution — all defined pixel-wise or via kernels.

Quantization: Converting continuous intensity to discrete levels. More levels = smoother images but more memory.

1.2 Color Spaces: RGB, BGR, HSV, LAB, YCrCb

RGB (Additive Color Model): - Red, Green, Blue channels combine additively to produce color. - In 8-bit: each channel 0-255. White = (255,255,255), Black = (0,0,0). - **Adding same constant c to all channels**: increases brightness uniformly. Hue is preserved (ratios unchanged) unless clipping occurs at 255. Saturation roughly preserved. Value/lightness increases. - **OpenCV loads as BGR by default** (historical reason). Must convert with `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before displaying with matplotlib. Otherwise colors appear swapped (sky reddish, skin bluish).

HSV (Hue-Saturation-Value): - **Hue** (0-179 in OpenCV, 0-360 in theory): dominant wavelength / color type. Pure red ≈ 0/180, green ≈ 60, blue ≈ 120. - **Saturation** (0-255): purity of color. 0 = gray (no color), 255 = fully saturated. - **Value** (0-255): brightness. 0 = black regardless of hue. - **Critical for color tracking**: lighting changes mostly affect V, leaving H relatively stable. Thresholding by H isolates specific colors regardless of brightness. - Example: track a red ball under varying lighting → threshold H near 0/180, ignore V variations.

LAB (CIELAB / L*a*b): - *L*: lightness (0 = black, 100 = white). - **A***: green ↔ red opponent dimension.* - ***B***: blue ↔ yellow opponent dimension. - **Perceptually uniform**: equal Euclidean distance in LAB ≈ equal perceived color difference by human visual system. - **Best for histogram equalization on color images**: convert BGR→LAB, equalize **L only**, convert back. This preserves hue and saturation while enhancing contrast.

Why NOT equalize RGB independently? - Histogram equalization is a non-linear intensity remapping. - Applying different remappings to R, G, B independently destroys the relative ratios between channels. - These ratios define hue. Result: unnatural, oversaturated, posterized colors. - **Always use LAB for color HE**.

YCrCb: - Y = luminance (like grayscale). Cr = red chrominance. Cb = blue chrominance. - Used in JPEG compression and video codecs because chrominance can be subsampled (human eye less sensitive to color detail). - Also useful for skin detection (skin tones cluster in specific Cr/Cb ranges).

1.3 Image Acquisition & Sensors

Digital Camera Pipeline: 1. **Light hits sensor** (photons → electrons via photoelectric effect) 2. **Analog-to-digital conversion** (ADC): converts analog voltage to digital values 3. **Demosaicing**: Bayer pattern → full RGB 4. **Color correction & white balance** 5. **Gamma correction**: linear sensor response → perceptually uniform display 6. **Compression** (optional): JPEG (lossy), PNG (lossless), TIFF (uncompressed)

CCD vs CMOS sensors: | Property | CCD | CMOS | |—|—|—| | Readout | Serial (all pixels through one amp) | Parallel (each pixel has own amp) | | Noise | Lower | Higher (improving) | | Speed | Slower | Faster | | Power | Higher | Lower | | Cost | Expensive | Cheap | | Applications | Scientific, medical, astronomy | Consumer cameras, phones, webcams |

Bayer filter: - Mosaic pattern: 50% green, 25% red, 25% blue (human eye most sensitive to green). - Each pixel records only ONE color. - **Demosaicing** interpolates missing colors from neighboring pixels. - Alternative: Foveon sensor (stacked RGB layers, no demosaicing needed).

Dynamic range (DR): ratio of max to min measurable intensity. - 8-bit image: theoretical DR = 255:1 (48 dB). - Modern cameras: 12-14 bit raw = 4096:1 to 16384:1 (72-84 dB). - Human eye: ~20 stops (1,000,000:1). - **HDR imaging**: capture multiple exposures (bracketing) and merge to extend DR.

Gamma correction: - Displays are non-linear: `output = input^γ` where $\gamma \approx 2.2$. - Camera applies inverse gamma ($\gamma \approx 0.45$) so that displayed image looks correct. - Without gamma correction, images would look too dark on displays.

Color temperature & white balance: - Different light sources have different spectra (sunlight ≈ 5500K, tungsten ≈ 3200K). - White balance adjusts R, G, B gains so that neutral gray appears neutral. - Incorrect white balance → color cast (image too blue or too yellow).

1.4 Convolution Theory

Definition: For 2D image `f` and kernel `h`:

$$(f * h)(i, j) = \Sigma_m \Sigma_n f(m, n) \cdot h(i-m, j-n)$$

Properties: - **Commutative**: $f * h = h * f$ - **Associative**: $(f * h_1) * h_2 = f * (h_1 * h_2)$ - **Distributive**: $f * (h_1 + h_2) = f*h_1 + f*h_2$ - **Shift-invariant**: shifting input shifts output by same amount

Cross-correlation vs Convolution: - In signal processing, convolution flips the kernel. - In deep learning / image processing, “convolution” usually means cross-correlation (no flip) because the kernel is learned anyway. The flip doesn’t matter for learnable filters.

Border handling strategies: - **Constant / Zero padding**: pad with 0. Most common in DL (keeps spatial dimensions with “same” padding). - **Replicate / Clamp padding**: pad with edge value. Common in traditional CV. Prevents dark borders. - **Reflect padding**: mirror the edge pixels. `...3 2 1 | 1 2 3 | 3 2 1...` - **Wrap / Circular padding**: image wraps around like a torus. - **Valid / No padding**: only compute where kernel fits entirely. Output shrinks. - **Same padding**: pad so output size equals input size (when stride=1). - **Full padding**: maximum padding. Output larger than input.

Output size formulas: - No padding, stride 1: `(H − k + 1) × (W − k + 1)` - With padding `p`: `(H + 2p − k + 1) × (W + 2p − k + 1)` - With stride `s`: `⌊(H + 2p − k) / s⌋ + 1`

1.5 Linear Filters: Mean (Box) Filter

- Kernel: all values `1/(k²)`. Every pixel in neighborhood contributes equally.
- **Effect**: uniform blur. Simple and fast.
- **Problems**:
    1. In frequency domain, sinc function has **side lobes → ringing artifacts** (Gibbs phenomenon) around sharp edges.
    2. Blurs across edges equally.
    3. Creates boxy/blocky artifacts because all neighbors weighted equally.
- **Use case**: very fast blur when quality doesn't matter. Not recommended for most CV tasks.

1.6 Linear Filters: Gaussian Filter

- 2D Gaussian: `G(x,y) = (1 / (2πσ²)) · exp(−(x²+y²) / (2σ²))`
- Weights decrease smoothly with distance from center.

Advantages over mean filter: 1. **Natural weighting**: closer pixels matter more. Smoother, more natural blur without block artifacts. 2. **No sharp frequency cutoff**: Gaussian has no zeros in Fourier transform → **no ringing artifacts**. 3. **Separable**: `G(x,y) = G(x) · G(y)`. A 2D Gaussian blur can be done as two 1D passes. Complexity drops from `O(k²·N)` to `O(2k·N)` — dramatic for large kernels. 4. **Scale-space property**: applying two Gaussians with σ_1 and σ_2 sequentially is equivalent to one Gaussian with $\sigma = \sqrt{\sigma_1^2 + \sigma_2^2}$. This is mathematically elegant and useful for multi-scale analysis. 5. **Closer to physical blur models**: defocus, atmospheric blur are naturally Gaussian.

Kernel size rule: `k = 6σ + 1` (captures ~99.7% of distribution, i.e., ±3σ).

Example: 5×5 Gaussian with σ=1 applied to 100×100 image: - Direct: 25 multiplications/pixel × 10,000 = 250,000. - Separable: (5+5) multiplications/pixel × 10,000 = 100,000. - **Savings: 60% reduction**.

1.7 Non-Linear Filters: Median Filter

- **Order-statistic filter**: replaces pixel with **median** of neighborhood values.
- Non-linear because sorting is involved.
- **Best for salt & pepper (impulse) noise**: outliers (0 or 255) are at extremes of sorted list. The median is almost never an extreme value → outliers are completely ignored.
- **Edge-preserving**: does not create new pixel values, only selects existing ones from the neighborhood. Sharp edges remain sharp.
- **Computational cost**: `O(k² log k²)` for naive implementation. Can be optimized for small kernels.
- **Comparison with Gaussian**: Gaussian smooths noise but blurs edges. Median removes impulse noise without blurring edges.

1.8 Non-Linear Filters: Bilateral Filter

- Weight = **spatial Gaussian** × **intensity Gaussian**.
- Formula: `w(i,j) = exp(−||p_i − p_j||² / (2σ_s²)) · exp(−||I_i − I_j||² / (2σ_r²))`
    - First term: spatial distance (closer = more weight).

    - Second term: intensity difference (similar intensity = more weight).
- **Edge-preserving**: at an edge, pixels on the other side have very different intensities → their weights drop to near zero → they don't contribute to the blur.
- **Parameters**: `d` (diameter of neighborhood), `sigmaColor` (σ_r), `sigmaSpace` (σ_s).
- **Downside**: computationally expensive. Not separable in general. Much slower than Gaussian.
- **Use case**: denoising while preserving edges (e.g., smoothing skin in portrait retouching while keeping eye/sharp edge detail).

1.9 Gradient Operators: Sobel

- Approximates first derivative (gradient) of image intensity.
- Includes implicit smoothing (unlike pure derivative) → less noise-sensitive.
- Kernels:
    - `G_x = [[−1, 0, 1], [−2, 0, 2], [−1, 0, 1]]` (horizontal edges)
    - `G_y = [[−1, −2, −1], [0, 0, 0], [1, 2, 1]]` (vertical edges)
- **Gradient magnitude**: `|G| = √(G_x² + G_y²)`
- **Gradient direction**: `θ = atan2(G_y, G_x)`
- **Use case**: edge detection, computing gradients for Harris corner detector, HOG features.

1.10 Gradient Operators: Laplacian

- Discrete approximation of **second derivative**.
- Kernel: `[[0, −1, 0], [−1, 4, −1], [0, −1, 0]]`
- Sum = 0 → responds to **changes** (edges), not uniform regions.
- **Very sensitive to noise** because second derivative amplifies high-frequency noise.
- **Solution**: apply Gaussian smoothing first → **Laplacian of Gaussian (LoG)**.
- **Use case**: edge detection via zero-crossings, blob detection.

1.10a Canny Edge Detection

- Multi-stage algorithm for optimal edge detection.

Steps: 1. **Gaussian smoothing**: reduce noise (σ controls smoothing amount). 2. **Gradient computation**: apply Sobel to get magnitude and direction. 3. **Non-maximum suppression (NMS)**: thin edges by keeping only local maxima along gradient direction. 4. **Double thresholding**: - Strong edges: pixels with gradient > `high_threshold`. - Weak edges: pixels between `low_threshold` and `high_threshold`. 5. **Edge tracking by hysteresis**: weak edges connected to strong edges are kept; isolated weak edges are discarded.

Parameters: - `low_threshold` and `high_threshold`: typical ratio is 1:2 or 1:3. - Larger σ → fewer, smoother edges. Smaller σ → more, noisier edges.

Advantages over simple gradient thresholding: produces thin, continuous edges with minimal noise. The hysteresis step connects broken edge segments.

1.11 Noise Models in Digital Images

Noise Type	Nature	Mathematical Model	Best Filter
Gaussian	Additive, independent	`I_noisy = I + N(0, σ²)`	Gaussian or mean
Salt & Pepper	Impulse, random min/max	Random pixels → 0 or 255	**Median**

Noise Type	Nature	Mathematical Model	Best Filter
Speckle	Multiplicative	`I_noisy = I + I·N`	Adaptive / homomorphic
Poisson	Signal-dependent	Variance ∝ mean	Variance-stabilizing transform
Quantization	Discrete levels	Round to nearest level	Dithering

Key insight: Choose the filter that matches the noise model. Median on Gaussian noise = suboptimal. Gaussian on salt & pepper = leaves outliers.

1.12 Histogram Equalization: Global (Standard HE)

Goal: Spread intensity values to use the full dynamic range `[0, L−1]`, improving contrast.

Steps: 1. Compute histogram `h(v)` = count of pixels with intensity `v`. 2. Compute CDF: `cdf(v) = Σ_{i=0}^{v} h(i)`. 3. Normalize CDF: `cdf_norm(v) = cdf(v) / N` where `N` = total pixels. 4. Map: `v' = round((L−1) · cdf_norm(v))`. 5. Replace every pixel with intensity `v` by `v'`.

Mathematical guarantee: The output histogram is as uniform as possible (discretization prevents perfect uniformity).

When it helps: Low-contrast images where intensity range is compressed into a narrow band. Dark images become brighter. **When it hurts**: Images with good contrast already (may over-amplify noise). Medical images where specific tissue appearances should be preserved.

Effect on grayscale: improves global contrast. Stretches dynamic range.

Effect on RGB: **NEVER equalize channels independently**. This destroys relative ratios → unnatural colors. Use LAB and equalize L only.

1.13 CLAHE (Contrast Limited Adaptive Histogram Equalization)

- **Problem with global HE**: over-amplifies noise in nearly uniform regions (e.g., sky, walls).
- **Solution**: operate locally and clip histogram peaks.

Algorithm: 1. Divide image into `tileGridSize` tiles (e.g., 8×8). 2. For each tile, compute histogram. 3. **Clip histogram**: limit the height of each bin to `clipLimit`. Redistribute clipped mass uniformly. 4. Apply HE to each tile independently. 5. Interpolate between tile boundaries to avoid block artifacts.

Parameters: - `clipLimit`: threshold for contrast limiting. Higher = more contrast. Typical: 2.0–4.0. - `tileGridSize`: number of tiles in row and column. Too small → blocky; too large → behaves like global HE.

Best for: images with regions of very different brightness (medical images with bright and dark areas, photos with strong shadows).

Color image procedure: 1. Convert BGR → LAB. 2. Apply CLAHE to **L** channel only. 3. Convert LAB → BGR.

1.14 Image Pyramids & Multi-Scale Processing

Gaussian Pyramid: - Build by repeatedly: Gaussian blur ($\sigma = 1.0$ for downsampling by 2) + downsample by factor 2. - Each level is 1/4 the size (1/2 width × 1/2 height) of the previous. - Level 0 = original image. Level 1 = half size. Level 2 = quarter size. - **Applications**: - Multi-scale object detection (detect large objects at coarse levels, small at fine levels). - Coarse-to-fine optical flow (estimate motion at coarse level, refine at finer levels). - Image blending (Laplacian pyramid blending).

Laplacian Pyramid: - `L_i = G_i - expand(G_{i+1})` where `G_i` is Gaussian pyramid level i. - Each level represents the detail lost when downsampling to the next level. - **Properties**: sum of all Laplacian levels (plus the coarsest Gaussian) reconstructs the original image exactly. - **Applications**: - **Seamless image blending**: blend images at multiple scales to avoid visible seams. - **Image compression**: encode coarse approximation + detail levels. - **Special effects**: "ORB" effect in Photoshop.

Scale-space theory: - A signal represented at multiple scales reveals different structures. - Gaussian smoothing creates a continuous scale-space (parameterized by σ). - **Scale-space extrema**: local maxima/minima across scale correspond to blob centers (basis of SIFT).

1.15 Image Interpolation Methods

When warping or resizing, we need to sample at non-integer coordinates. The quality of interpolation directly affects the result.

Nearest neighbor: - Round `(x, y)` to closest integer pixel. - **Fastest** (no computation). - **Worst quality**: creates jagged edges, blocky artifacts, aliasing. - Use only when speed is critical and quality doesn't matter.

Bilinear interpolation: - For point `(x, y)`, find 4 surrounding pixels: `(x0,y0)`, `(x1,y0)`, `(x0,y1)`, `(x1,y1)`. - Interpolate horizontally first: `I(x,y0) = (1-α)·I(x0,y0) + α·I(x1,y0)` where `α = x - x0`. - Then vertically: `I(x,y) = (1-β)·I(x,y0) + β·I(x,y1)` where `β = y - y0`. - **Smooth, continuous** but not continuously differentiable at grid points. - **Standard choice** for most CV tasks. OpenCV default.

Bicubic interpolation: - Uses 16 neighboring pixels (4×4 grid) and cubic polynomials. - Computes weighted sum with weights based on cubic interpolation kernel. - **Smoother** than bilinear, less aliasing. - **Slower** (4× more samples than bilinear). - Better for enlarging images where quality matters.

Lanczos resampling: - Based on sinc function (ideal reconstruction for band-limited signals). - Uses larger neighborhood (e.g., 8×8 = 64 pixels for Lanczos-3). - **Highest quality** for image upscaling. - **Slowest** due to large neighborhood. - Often used in professional image editing.

Anti-aliasing when downsampling: - Simply subsampling (taking every Nth pixel) causes aliasing. - **Correct approach**: apply low-pass filter (Gaussian) BEFORE downsampling. - This removes frequencies above the Nyquist rate of the new resolution. - OpenCV `resize` with `INTER_AREA` for downsampling: computes pixel area relation (good for downscaling).

OpenCV defaults: - `warpAffine`/`warpPerspective`: bilinear (`INTER_LINEAR`). - `resize` for downscaling: `INTER_AREA` (area resampling). - `resize` for upscaling: `INTER_CUBIC` (cubic).

1.15a Image Quality Metrics

PSNR (Peak Signal-to-Noise Ratio):

$$PSNR = 10 \cdot \log_{10}(MAX^2 / MSE)$$

where `MAX` = maximum possible pixel value (255 for 8-bit), `MSE` = mean squared error between original and reconstructed image. - Higher PSNR = better quality. - Typical values: 30-50 dB for good quality. - **Limitation**:

poorly correlates with human perception. Two images with same PSNR can look very different.

SSIM (Structural Similarity Index):

$$SSIM(x,y) = (2\mu_x \mu_y + c1)(2\sigma_{xy} + c2) / ((\mu_x^2 + \mu_y^2 + c1)(\sigma_x^2 + \sigma_y^2 + c2))$$

- Compares luminance, contrast, and structure.
- Range: [-1, 1], where 1 = identical.
- **Better correlates with human perception** than PSNR.
- Computes locally over windows and averages.
- Used to evaluate denoising, compression, super-resolution.

When to use which: - Use **PSNR** for quick, objective numerical comparison (common in compression). - Use **SSIM** when you care about perceptual quality (common in restoration, super-resolution). - Neither is perfect: human evaluation remains the gold standard for image quality.

1.16 Optical Flow: Introduction & Assumptions

Optical flow = apparent motion of brightness patterns in an image sequence. It is a 2D vector field `(u, v)` assigned to each pixel.

Three core assumptions: 1. **Brightness constancy**: `I(x, y, t) = I(x+dx, y+dy, t+dt)` — pixel brightness doesn't change between frames. 2. **Small motion**: `dx, dy` are small enough for first-order Taylor approximation. 3. **Spatial coherence**: neighboring pixels move similarly (flow is smooth locally).

Brightness Constancy Equation (BCE): Taking Taylor expansion of brightness constancy:

$$I_x \cdot u + I_y \cdot v + I_t = 0$$

where $I_x = \partial I/\partial x$, $I_y = \partial I/\partial y$ (spatial gradients), $I_t = \partial I/\partial t$ (temporal derivative), and (u, v) is the flow vector.

This is ONE equation with TWO unknowns → under-constrained. We need additional constraints.

1.17 Lucas-Kanade (Sparse Optical Flow)

Key assumption: Flow `(u, v)` is **constant** in a local window of size `w×w`.

This gives an over-determined system (many pixels, same `u, v`):

$$\begin{bmatrix} \Sigma I_x^2 & \Sigma I_x \cdot I_y \\ \Sigma I_x \cdot I_y & \Sigma I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -\Sigma I_x \cdot I_t \\ -\Sigma I_y \cdot I_t \end{bmatrix}$$

All sums are over the local window.

Matrix must be invertible → requires **textured region** (both eigenvalues of the structure tensor must be large).

Aperture problem: In a small window on an edge, only the flow component **perpendicular** to the edge is observable. The component parallel to the edge is ambiguous. Lucas-Kanade fails here because the matrix is singular (one eigenvalue ≈ 0).

Pyramid implementation (`calcOpticalFlowPyrLK`): - Builds Gaussian pyramid of both images. - Computes flow from **coarsest to finest** level. - At each level, the flow estimate from the coarser level warps the finer level. - This handles **larger displacements** that violate the small-motion assumption.

Application: tracking good features (corners) across video frames. Real-time performance.

1.18 Why Optical Flow ≠ True Motion (Failure Cases)

1. **Brightness changes without motion**: moving shadows, specular reflections, changing illumination. Flow detects motion where there is none.
2. **No texture / uniform regions**: rotating uniform white sphere under static lighting → zero brightness change → **zero optical flow**, despite physical rotation.
3. **Aperture problem**: motion along an edge is ambiguous in a small window.
4. **Occlusions**: newly exposed background has no corresponding pixel in the previous frame. No valid flow.
5. **Transparency**: seeing through a moving glass window confuses motion estimation (two motions visible).
6. **Large motion**: violates small-motion assumption unless pyramid is used.

Dense vs Sparse: - **Dense** (Farneback, TV-L1): computes flow for every pixel. Computationally expensive. Better quality but slower. - **Sparse** (Lucas-Kanade): computes flow only at salient points (corners). Much faster. Sufficient for tracking.

For real-time tracking: sparse optical flow is preferred.

---

2. FEATURES, LOCAL FEATURES, AND WARPING

2.1 Why Detect Features? What Makes a Good Feature?

Feature: A distinctive, repeatable, local pattern in an image.

Ideal properties of a local feature: 1. **Repeatability**: same scene under different conditions → same features detected. 2. **Distinctiveness**: features should be easy to tell apart. Not all corners are equally distinctive. 3. **Locality**: features should be local (small region) so they are robust to occlusion. 4. **Quantity**: enough features to cover the scene and enable reliable matching. 5. **Accuracy**: precise localization (sub-pixel accuracy desirable). 6. **Efficiency**: fast to compute and match (critical for real-time applications).

Why corners over edges or flat regions? - **Flat regions**: no intensity change in any direction → ambiguous, no information. - **Edges**: strong intensity change in one direction only → ambiguous because you can slide along the edge without changing appearance. - **Corners**: strong intensity changes in **two or more directions** → locally unique, unambiguous.

2.2 Harris Corner Detector

Mathematical derivation: The auto-correlation function measures how much patch intensity changes when shifted by `(u, v)`:

`E(u,v) = Σ_w [I(x+u, y+v) − I(x,y)]²`

Using first-order Taylor: `I(x+u, y+v) ≈ I(x,y) + u·I_x + v·I_y`. So: `E(u,v) ≈ [u, v] · M · [u, v]^T` where `M = Σ_w [[I_x², I_x·I_y], [I_x·I_y, I_y²]]`.

Structure tensor / auto-correlation matrix `M`: - Sum of outer products of gradients in a window. - Symmetric 2×2 matrix.

Eigenvalues of M determine local structure: | λ_1 | λ_2 | Interpretation | Region Type | |—|—|—|—| | Small | Small | No gradient | Flat | | Large | Small | Gradient in one direction only | Edge | | Large | Large | Gradient in all

directions | Corner |

Harris response:

$R = \det(M) - k \cdot \text{trace}(M)^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$

- k = empirical constant, typically 0.04 to 0.06.
- $R > 0$ and large → **corner** (both eigenvalues large, product dominates).
- $R < 0$ and large magnitude → **edge** (one eigenvalue dominates, squared sum larger than product).
- $|R| \approx 0$ → **flat region** (both eigenvalues small).

Why R is negative on edges: $\det = \lambda_1\lambda_2$ is small. $\text{trace}^2 = (\lambda_1+\lambda_2)^2 \approx \lambda_1^2$ (one eigenvalue dominates). For large λ_1, $k \cdot \text{trace}^2$ exceeds $\det \rightarrow R < 0$.

Changing sensitivity: - Lower k → less penalty for edges → more corners detected. - Lower threshold (e.g., $0.001 \cdot R\_max$ instead of $0.01 \cdot R\_max$) → more corners. - Smaller `minDistance` → corners can be closer together (more total). - Smaller `blockSize` → analyzes smaller neighborhoods → finer corners. - For fewer corners: do the opposite.

Non-maximum suppression: after computing R across the image, keep only local maxima to remove clustered duplicate corners.

2.3 Shi-Tomasi Corner Detector (goodFeaturesToTrack)

- Improvement over Harris.
- Uses $R = \min(\lambda_1, \lambda_2)$ directly.
- **More stable** because it doesn't combine eigenvalues through the Harris formula that depends on the arbitrary choice of k.
- If $\min(\lambda_1, \lambda_2) >$ threshold → corner accepted.
- Parameters: `maxCorners`, `qualityLevel` (fraction of best corner), `minDistance`, `blockSize`.
- OpenCV: `cv2.goodFeaturesToTrack()`.

2.4 FAST Corner Detector

- **Features from Accelerated Segment Test**.
- Designed for **speed** — one of the fastest corner detectors.

Algorithm: 1. Consider a Bresenham circle of radius 3 around pixel p (16 pixels on the circle). 2. Compare intensity of these 16 pixels to $I(p)$ + threshold t (brighter) or $I(p) - t$ (darker). 3. If there exist N contiguous pixels on the circle that are ALL brighter or ALL darker by threshold t, then p is a corner. 4. N is typically 9, 10, 11, or 12. Higher N = stricter = fewer corners. 5. Apply non-maximum suppression to remove clustered responses.

Advantages: extremely fast (no derivatives, no eigenvalues), suitable for real-time. **Disadvantages**: not scale-invariant, no orientation, less robust to noise than Harris. **Used in**: ORB, real-time AR/SLAM.

2.5 Blob Detection & SIFT

What is a blob? - A region that stands out from surroundings in terms of intensity or texture. - Detected at multiple scales (unlike corners which are point features).

SIFT (Scale-Invariant Feature Transform):

Operates on grayscale — keypoint detection relies on intensity gradients, not color. Running on R, G, B independently would produce redundant keypoints.

Four main steps:

1. **Scale-space extrema detection**:
   - Build **Difference-of-Gaussian (DoG) pyramid**.
   - DoG approximates Laplacian of Gaussian (LoG) but is computationally cheaper.
   - Find local maxima/minima across scales → these are blob-like structures at different sizes.
2. **Keypoint localization**:
   - Fit 3D quadratic to refine sub-pixel location, scale, and orientation.
   - Reject low-contrast keypoints (unstable) and edge responses (using Harris-like threshold on ratio of principal curvatures).
3. **Orientation assignment**:
   - Compute gradient orientation histogram in neighborhood around keypoint.
   - Peak(s) of histogram = dominant orientation(s).
   - Assign orientation to keypoint → makes descriptor **rotation-invariant**.
4. **Descriptor computation**:
   - Take 16×16 neighborhood around keypoint.
   - Divide into 4×4 cells.
   - Each cell: 8-bin orientation histogram.
   - Total: **4 × 4 × 8 = 128 dimensions**.
   - Normalize to unit length → illumination invariant.

Properties: - **Invariant to**: scale (DoG pyramid), rotation (orientation assignment), illumination (gradient-based, normalization). - **NOT invariant to**: affine transformations (strong perspective foreshortening), viewpoint changes for non-planar objects. - **Patent**: expired in most countries; freely usable. - **Robust but slow**: not suitable for real-time on mobile devices.

2.6 ORB (Oriented FAST and Rotated BRIEF)

- Created as a **free, fast alternative to SIFT/SURF**.

Detection: Uses **FAST** (extremely fast corner detection).

Description: Uses **BRIEF** (Binary Robust Independent Elementary Features): - Binary descriptor: each bit is result of intensity comparison between two pixel locations around keypoint. - Example: bit_i = 1 if I(p_a) > I(p_b), else 0. - Very fast to compute (just pixel comparisons).

Orientation: ORB adds orientation to FAST by measuring **intensity centroid** of the patch and computing dominant angle. Then **rotates the BRIEF pattern** accordingly → **steered BRIEF**.

Matching: Uses **Hamming distance** (count of differing bits). - Can be computed extremely fast with XOR + popcount (CPU instruction). - Much faster than SIFT's L2 distance.

Comparison with SIFT: - ORB: faster by ~2 orders of magnitude, binary descriptor, less memory. - ORB: less robust to scale changes and large rotations. - SIFT: more robust, patented (expired), 128D float descriptor. - **Best use**: real-time applications (AR on mobile, SLAM). SIFT for accuracy-critical offline tasks.

2.7 Feature Matching: Distance Metrics & Algorithms

Distance metrics: - **L2 (Euclidean)**: $d(a,b) = \sqrt{(\Sigma(a_i - b_i)^2)}$. Used for SIFT float descriptors. - **Hamming**: `d(a,b) = count of differing bits`. Used for ORB binary descriptors. Computed with XOR + popcount.

Matching algorithms: - **Brute Force (BF)**: compare every descriptor in image A to every descriptor in image B. $O(N \cdot M)$. Precise but slow for large datasets. Good for small datasets or when precision is paramount. - **FLANN (Fast Library for Approximate Nearest Neighbors)**: uses approximate search structures. - For SIFT: **KD-trees** (k-dimensional trees). $O(N \log M)$. - For ORB/High-dim: **k-means trees** or **LSH (Locality Sensitive Hashing)**. - Much faster but occasional approximate errors (might miss true nearest neighbor). - Preferred for large datasets (thousands+ of features).

2.8 Lowe's Ratio Test

Problem: Many matches are ambiguous. A descriptor might match equally well to two different points in the target image (repetitive patterns, textureless regions).

Method: 1. For each query descriptor, find the **2 nearest neighbors** in target image. 2. Compute ratio: `r = d(best) / d(second)`. 3. Accept match only if `r < 0.7` (or 0.8, depending on desired precision/recall trade-off).

Intuition: If best and second-best are very similar, the match is ambiguous → likely incorrect → reject. - This removes many false matches before geometric verification. - Typical result: retains ~60-90% inliers (still some outliers, which is why RANSAC is needed).

2.9 RANSAC (Random Sample Consensus)

Problem: Even after Lowe's ratio test, feature matching produces **outliers** (incorrect correspondences). Least-squares fitting assumes all points are valid → outliers severely distort the result.

RANSAC algorithm: 1. Select **minimum sample size** `s`: - 4 for homography, 3 for affine, 2 for line, 8 for fundamental matrix. 2. Compute model hypothesis `M` from the sample using **DLT (Direct Linear Transform)**. 3. Count **inliers**: points consistent with `M` within threshold `t`. 4. Repeat for `N` iterations. 5. Keep model with **maximum inliers**. 6. Re-estimate model using **all inliers** for a refined solution.

Number of iterations formula:

```
N = log(1 − p) / log(1 − w^s)
```

where: - `p` = desired success probability (e.g., 0.99). - `w` = estimated fraction of inliers (e.g., 0.5 if 50% are inliers). - `s` = sample size.

Example: For homography (`s=4`), `w=0.5`, `p=0.99`:

```
N = log(0.01) / log(1 − 0.5^4) = log(0.01) / log(0.9375) ≈ 4.6 / 0.0645 ≈ 71 iterations
```

Why essential for homography: Feature matching typically yields 60-90% inliers after ratio test, but even 10% outliers would severely bias least-squares. RANSAC is robust to up to **50% outliers**.

OpenCV: `cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, ransacReprojThreshold)`

2.10 Geometric Transformations: Translation, Euclidean, Similarity

Translation: - `x' = x + t_x`, `y' = y + t_y` - 2 DOF. Preserves everything (orientation, size, shape).

Euclidean (Rigid): - `x' = x·cosθ − y·sinθ + t_x` - `y' = x·sinθ + y·cosθ + t_y` - 3 DOF (rotation + translation). - Preserves: lengths, angles, areas, parallel lines. - Maps triangles to congruent triangles.

Similarity: - Adds uniform scaling to Euclidean. - 4 DOF (scale + rotation + translation). - Preserves: angles, ratios of lengths, ratios of areas. - Maps triangles to similar triangles.

2.11 Affine Transformation

Matrix form:

```
[x']   [a11 a12 a13] [x]
[y'] = [a21 a22 a23] [y]
[1 ]   [ 0   0   1 ] [1]
```

- **6 degrees of freedom**.
- Composed of: translation, rotation, uniform/non-uniform scaling, shear.
- **Preserves**: straight lines, parallel lines, ratios of lengths on parallel lines, ratios of areas.
- **Does NOT preserve**: absolute lengths, angles (unless pure rotation), areas (unless no scaling).
- Maps parallelograms to parallelograms.
- **3 point pairs** needed to estimate.

When to use: simple 2D manipulations (rotate/scale a logo), when scene is far enough that perspective effects are negligible, or when you need stability with fewer parameters than homography.

2.12 Projective / Homography Transformation

Matrix form:

```
[x']   [h11 h12 h13] [x]
[y'] = [h21 h22 h23] [y]
[w']   [h31 h32 h33] [1]
```

- Normalized: `x' = x'/w'`, `y' = y'/w'`.
- **8 degrees of freedom** (3×3 matrix up to scale; `h33` often set to 1).
- **Preserves**: straight lines.
- **Does NOT preserve**: parallelism (parallel lines converge at vanishing points), ratios of lengths, ratios of areas.
- Maps quadrilaterals to quadrilaterals.
- **4 point pairs** needed (no 3 collinear, otherwise degenerate).

When to use: - Same plane viewed from different angles → homography relates the two views. - Camera pure rotation about its center (no translation) → homography relates any two views regardless of scene geometry. - Image stitching (panoramas). - Perspective correction (document scanning, building photos). - AR overlay on planar surfaces (billboards, markers).

When NOT to use: general 3D scene with camera translation → use **epipolar geometry** (fundamental/essential matrix) because points at different depths move differently.

2.13 Image Warping: Forward vs Backward

Forward warping: - For each source pixel `(x, y)`, compute destination `(x', y')` via transformation. - Copy pixel value to destination. - **Problems**: - **Gaps**: when compression occurs, multiple source pixels map to same destination → some destination pixels get no value. - **Overlaps**: when expansion occurs, need to decide which source pixel wins (or blend).

Backward warping (preferred): - For each destination pixel `(x', y')`, compute source location `(x, y)` via **inverse transformation**. - Sample source image at `(x, y)`. If non-integer, use interpolation (bilinear standard). - **Advantages**: - **No gaps**: every destination pixel gets a value. - Simple and clean. - **Disadvantage**: requires computing the inverse transform.

OpenCV: `cv2.warpAffine` and `cv2.warpPerspective` use **backward warping** with bilinear interpolation by default.

2.14 Image Blending & Compositing

Alpha blending: `I_out = α·I_fg + (1−α)·I_bg` - `α` = alpha mask (0 to 1). Used for smooth transitions.

Poisson blending: solves Poisson equation to seamlessly blend source into target while preserving gradients. - Better than alpha blending at boundaries because it matches gradients, not just colors.

Pyramid blending: blend images at multiple resolutions using Laplacian pyramids. - Avoids seams by blending low frequencies over large regions and high frequencies locally.

---

3. SEGMENTATION AND MOTION

3.1 Image Segmentation Overview

Goal: Partition an image into meaningful regions (objects, boundaries).

Types of segmentation: - **Semantic segmentation**: classify every pixel into a category class (e.g., “person”, “car”, “road”). All instances of same class share label. - **Instance segmentation**: semantic + distinguish individual objects (person 1, person 2). - **Panoptic segmentation**: combines semantic (for “stuff”: sky, grass, road) and instance (for “things”: people, cars) into unified output.

3.2 Global Thresholding

- Single threshold `T` for entire image.
- `pixel > T` → foreground (white), else background (black).
- **Only works** when:
  - Foreground and background have clearly separable intensities.
  - Illumination is uniform across the image.
  - Histogram is bimodal (two clear peaks).
- **Simple and fast** but very limited applicability.

3.3 Otsu’s Method

- **Automatic** threshold selection. No manual tuning needed.
- **Principle**: maximize **between-class variance** (equivalently, minimize within-class variance).

Formula:

$$\sigma^2_b(T) = \omega_0(T) \cdot \omega_1(T) \cdot [\mu_0(T) - \mu_1(T)]^2$$

where: - ω_0, ω_1 = class probabilities (fraction of pixels below/above T). - μ_0, μ_1 = class mean intensities.

Algorithm: 1. Compute image histogram. 2. For every possible threshold `T` from 0 to 255: - Split pixels into two classes. - Compute $\sigma^2_b(T)$. 3. Select `T` that maximizes σ^2_b.

Assumptions: - Histogram is roughly **bimodal**. - Classes have roughly equal sizes (works less well with extreme imbalances).

When it fails: - Unimodal histogram (no clear separation). - Modes overlap heavily. - Uneven illumination across image. - Noisy histogram (needs pre-smoothing).

3.4 Adaptive Thresholding

- **Problem**: global threshold fails for uneven illumination (e.g., document with shadow).
- **Solution**: compute threshold locally for each pixel based on its neighborhood.

OpenCV methods: - `ADAPTIVE_THRESH_MEAN_C`: `T(x,y) = mean(neighborhood) − C` - Simple average of neighborhood minus constant. - `ADAPTIVE_THRESH_GAUSSIAN_C`: `T(x,y) = GaussianWeightedMean(neighborhood) − C` - Gaussian-weighted average of neighborhood minus constant. - Gives more weight to center pixels → smoother thresholds.

Parameters: - `blockSize`: size of local neighborhood (must be odd). Larger = more global. Typical: 11, 21, 31. - `C`: constant subtracted from mean. Compensates for bias. Typical: 2, 5, 10.

Best for: scanned documents with shadows, uneven lighting, vignetting.

Pre-processing: apply mild Gaussian blur before thresholding to reduce noise and improve stability.

3.5 K-Means Segmentation

- Treats segmentation as **clustering in feature space**.
- Each pixel = point in 3D RGB space (or HSV, LAB).
- Cluster pixels into `k` groups using K-Means algorithm.
- Each pixel is replaced by its cluster center color.

Properties: - **Fast**: K-Means converges quickly (usually < 20 iterations). - **No spatial coherence**: clusters purely by color. Two adjacent pixels with slightly different colors may end up in different clusters → fragmented regions. - Choice of `k` is critical. Too small = distinct regions merged. Too large = oversegmentation. - Converges to **local minimum** → sensitive to initialization. Run multiple times with different seeds.

Improvements: - **Spatial K-Means**: add `(x, y)` coordinates as additional dimensions. Balances color similarity and spatial proximity. - **Mean Shift**: no need to specify `k`. More robust but slower.

3.6 Mean Shift Segmentation

- **Mode-seeking clustering** in feature space.
- Starts at each data point and iteratively shifts toward the densest region (mode) in its neighborhood.
- **Bandwidth** parameter defines neighborhood size.

- **Advantages**:
  - No need to specify k (number of clusters).
  - More robust to outliers than K-Means.
- **Disadvantages**:
  - Much slower than K-Means (doesn't scale well to large images).
  - Bandwidth selection is non-trivial.

3.7 GrabCut Segmentation

- **Interactive / semi-automatic** segmentation.
- Combines **graph cuts** with **Gaussian Mixture Models (GMMs)**.

Algorithm: 1. **User initializes**: draws a rectangle around the object, or provides a rough mask. 2. **Initial labeling**: pixels outside rectangle = definite background. Pixels inside = probable foreground. 3. **GMM learning**: model foreground and background color distributions with separate GMMs. 4. **Graph cut**: build a graph where pixels are nodes. Edges encode similarity and penalty for cutting. - Solve minimum cut → optimal segmentation given current models. 5. **Iterate**: refine GMMs based on new labels, re-run graph cut. Repeat until convergence.

Four labels: - **Definite background**: fixed, never changes. - **Probable background**: can change to foreground during iterations. - **Probable foreground**: can change to background during iterations. - **Definite foreground**: fixed, never changes.

Why "probable" categories are needed: the initial rectangle is imprecise. The algorithm needs flexibility to refine boundaries during optimization.

Very accurate when foreground/background have good color contrast.

3.8 Graph-Cut Optimization

- Image modeled as a graph where pixels are nodes.
- **Source node** represents foreground. **Sink node** represents background.
- **Edge weights** encode:
  - Similarity between adjacent pixels (higher weight = more likely to have same label).
  - Likelihood of pixel belonging to foreground/background (from GMM or user input).
- **Minimum cut** separates source from sink → optimal segmentation.
- Polynomial-time solvable (max-flow/min-cut algorithms).

3.9 Traditional vs Deep Learning Segmentation

Traditional methods (thresholding, K-Means, GrabCut, Active Contours, Watershed): - Rely on **hand-crafted features**: intensity, color, edges, texture. - Use manually designed rules and thresholds. - **Cannot learn or improve with more data**. - **Struggle with**: - Complex scenes with many objects. - Object variability (different poses, shapes, colors). - Occlusions. - Lighting changes. - Do not generalize well across domains.

Why semantic segmentation is especially hard for traditional methods: - Pixel-level decisions based only on local appearance are ambiguous. - A pixel that looks like "grass" might actually be "green car" without global context. - Need to understand "what" and "where" simultaneously.

Deep Learning methods (FCN, U-Net, DeepLab, Mask R-CNN): - **Learn hierarchical features automatically** from data. - **Encoder-decoder architectures**: - Encoder: captures global context (WHAT objects are present) via

repeated pooling. - Decoder: recovers spatial resolution (WHERE they are) via upsampling. - **Skip connections** preserve fine boundaries lost during pooling. - **Massively superior** on semantic and instance segmentation benchmarks. - Improve with more data and compute.

3.10 Morphological Operations

Basic operations on binary images:

Erosion: - Pixel remains foreground only if ALL pixels in structuring element are foreground. - **Effect**: shrinks objects, removes small noise, separates touching objects.

Dilation: - Pixel becomes foreground if ANY pixel in structuring element is foreground. - **Effect**: grows objects, fills small holes, connects broken regions.

Opening = Erosion + Dilation: - Removes small noise objects and thin protrusions. - Preserves approximate size of larger objects.

Closing = Dilation + Erosion: - Fills small holes and gaps. - Preserves approximate size.

Structuring element: shape (disk, rectangle, cross) and size determine effect.

3.11 Distance Transform & Watershed

Distance transform: - For each foreground pixel, compute distance to nearest background pixel. - Result: peaks (local maxima) correspond to centers of objects.

Watershed algorithm: - Treats distance transform (or gradient magnitude) as a topographic surface. - **Markers**: local maxima of distance transform = starting points (object centers). - **Flooding**: simulate water rising from markers. - **Watershed lines**: boundaries form where floods from different markers meet. - These lines separate touching/overlapping objects.

Pipeline for touching cells: 1. Threshold to get binary mask. 2. Distance transform. 3. Find local maxima (markers). 4. Apply watershed. 5. Result: individual cell segmentations.

3.12 Optical Flow: Dense Methods

Farneback optical flow: - Polynomial expansion to approximate neighborhood with quadratic polynomial. - Computes dense flow field. - Slower than Lucas-Kanade but more accurate for smooth motion.

TV-L1 (Total Variation L1): - Variational method minimizing total variation regularization with L1 data term. - Handles occlusion boundaries well. - Very slow, not real-time.

Comparison: - **Sparse** (Lucas-Kanade): fast, tracks corners, sufficient for tracking. - **Dense** (Farneback, TV-L1): computes flow everywhere, better for motion analysis, much slower.

---

4. CONVOLUTIONAL NEURAL NETWORKS

4.1 Motivation: Why CNNs over MLPs for Images?

MLP (Fully Connected Network) limitations: - A 1000×1000 RGB image has 3,000,000 input pixels. - Single hidden layer with 1000 neurons: 3,000,000 × 1000 = **3 billion parameters**. - **Computationally infeasible**. - **Destroys spatial structure**: flattening loses all 2D relationships (adjacent pixels become distant in the vector). - **No translation invariance**: if object moves, different neurons must respond. No weight sharing.

CNN advantages: 1. **Sparse connectivity**: each neuron connects only to a local receptive field (e.g., 3×3 patch), not the entire image. 2. **Weight sharing**: the same filter (set of weights) is applied across the entire image. If a cat detector works at position (10,10), it also works at (100,100). This provides **translation invariance** and dramatically reduces parameters. 3. **Hierarchical feature composition**: early layers detect edges, mid layers combine edges into textures/parts, late layers combine parts into objects. 4. **Preserve spatial structure**: 2D topology maintained throughout convolutional layers.

4.2 Convolutional Layer: Detailed Operation

Operation: Slide `C_out` learnable filters (kernels) over the input. - Each filter has shape `(C_in, K_h, K_w)`. - At each spatial position, compute dot product between filter and input patch, add bias. - Each filter produces one **feature map** (output channel).

Output shape:

```
H_out = ⌊(H_in + 2·P_h − K_h) / S_h⌋ + 1
W_out = ⌊(W_in + 2·P_w − K_w) / S_w⌋ + 1
C_out = number of filters
```

Parameters:

```
Params = (K_h · K_w · C_in + 1) · C_out
```

The `+1` is the **bias term per filter**.

Receptive field: region in the original image that influences a neuron's output. - With stride 1, no dilation: `RF_L = RF_{L−1} + (K_L − 1) · J_{L−1}` where `J` is cumulative stride. - **Stack of three 3×3 convolutions = 7×7 receptive field**.

Cross-correlation vs true convolution: - True convolution flips the kernel. Cross-correlation does not. - In deep learning, "convolution" usually means cross-correlation because kernels are learned — the flip doesn't matter.

4.3 Strided Convolutions & Transposed Convolutions

Strided convolution: - Stride `S > 1` skips pixels during sliding → downsampling. - Example: stride 2 reduces spatial dimensions by ~half. - **Learnable downsampling**: unlike max pooling, strided conv has parameters. - Replacing max pool with strided conv increases parameters but may learn better downsampling.

Transposed convolution (Deconvolution / Fractionally-strided convolution): - Used for upsampling in decoder networks. - Learns to upsample by inserting zeros between input elements and applying convolution. - Has learnable parameters unlike fixed upsampling (nearest neighbor, bilinear). - **Not a true inverse** of convolution. Artifacts possible (checkerboard patterns). - Solutions: resize-conv (upsample with bilinear then conv), sub-pixel convolution.

4.4 Activation Functions

ReLU (Rectified Linear Unit): - `f(x) = max(0, x)` - **Pros**: no saturation for positive inputs (gradient = 1). Computationally cheap (just thresholding). Induces sparsity (~50% of neurons off). - **Cons**: "Dying ReLU" — if a neuron's weights are initialized badly or learning rate is too high, it can get stuck with all negative inputs, always outputting 0. No gradient flows through it.

Leaky ReLU: - `f(x) = max(α·x, x)` with small α (e.g., 0.01). - Allows small negative gradient → fixes dying ReLU.

PReLU (Parametric ReLU): α is learned per neuron.

Softmax: - `σ(z)_i = e^{z_i} / Σ_j e^{z_j}` - Used for **multi-class classification output**. - Converts logits to probability distribution (sums to 1). - Amplifies differences (largest logit gets much larger probability).

Sigmoid: - `σ(x) = 1 / (1 + e^{−x})` - Used for **binary classification** and GAN discriminator output. - Saturates at extremes → vanishing gradient problem. That's why ReLU replaced sigmoid in hidden layers.

4.5 Pooling Layers

Max Pooling: - Output = maximum value in `k×k` window. - **Provides translation invariance**: small shifts in input don't change output. - **No trainable parameters**. - Downsampling reduces computation and memory. - Standard in classification networks (VGG, AlexNet).

Average Pooling: - Output = mean value in window. - Smoother than max pooling but less selective. - Sometimes preferred in very deep networks or dense prediction.

Global Average Pooling (GAP): - Average each feature map to a **single value**. - Replaces fully connected layers entirely. - **Massive parameter reduction**: e.g., 512 channels → 512 values instead of millions of FC parameters. - Less overfitting, accepts variable input sizes. - Used in ResNet, MobileNet, Network in Network.

4.6 Dropout

- **During training**: randomly sets fraction `p` (e.g., 0.5) of neuron outputs to 0.
- **Prevents co-adaptation**: neurons cannot rely on specific other neurons always being present.
- **Forces robust representations**: network must learn redundant, distributed features.
- **At test time (inference)**: ALL neurons are active. Weights are scaled by `(1−p)` (or equivalent) to maintain expected value.
- **Only applied during training**: `model.train()` enables dropout; `model.eval()` disables it.
- **Typical placement**: after fully connected layers (where overfitting is worst). Less common after conv layers.
- Variants: Spatial Dropout (drops entire feature maps), DropConnect (drops weights instead of activations).

4.7 Batch Normalization

- **Problem (Internal Covariate Shift)**: as parameters of previous layers change, the distribution of inputs to current layer changes. This requires lower learning rates and careful initialization.

Operation: 1. Normalize per mini-batch: `x̂ = (x − μ_B) / √(σ²_B + ε)` 2. Scale and shift: `y = γ·x̂ + β` - γ (scale) and β (shift) are **learnable parameters**.

Benefits: 1. **Stabilizes training**: inputs to each layer have consistent distribution. 2. **Allows higher learning rates**. 3. **Reduces dependence on initialization**. 4. **Acts as regularizer**: noise from batch statistics has

regularizing effect (similar to dropout). 5. Often **reduces or eliminates need for dropout**.

At inference: uses running mean/variance computed during training (not batch statistics).

Placement: typically after convolution / before activation. Or before convolution (pre-activation ResNet).

4.8 Training Essentials

Loss Functions: - **Cross-Entropy**: $L = -\Sigma\ y_i \cdot \log(\hat{y}_i)$. Standard for classification. - In PyTorch, `nn.CrossEntropyLoss` combines `LogSoftmax` + `NLLLoss`. Do NOT apply softmax before this loss. - **MSE (Mean Squared Error)**: $L = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2$. Standard for regression. - **BCE (Binary Cross Entropy)**: for binary classification / binary segmentation.

Optimizers: - **SGD with Momentum**: $v_t = \beta \cdot v_{t-1} + \nabla L \quad w_t = w_{t-1} - lr \cdot v_t$ Momentum accumulates velocity. Helps escape shallow local minima and accelerates in consistent directions.

- **Adam (Adaptive Moment Estimation)**:
  - Maintains moving average of gradient (first moment, like momentum) and squared gradient (second moment, like RMSprop).
  - Adapts learning rate per parameter.
  - Default choice for many tasks. Converges faster than SGD initially.
- **AdamW**: decouples weight decay from gradient update. Better regularization than Adam with L2 penalty. Now recommended over Adam for many tasks.

Backpropagation pipeline: 1. **Forward pass**: input → network → compute prediction. 2. **Compute loss**: compare prediction to target. 3. **`loss.backward()`**: compute gradients via chain rule. 4. **`optimizer.zero_grad()`**: clear old gradients (PyTorch accumulates by default). 5. **`optimizer.step()`**: update weights using gradients.

Train / Validation / Test split: - **Training set**: update weights. Typically 70-80% of data. - **Validation set**: tune hyperparameters, detect overfitting, early stopping. NOT used for weight updates. 10-15%. - **Test set**: final unbiased evaluation. Used **ONLY ONCE** at the very end. 10-15%. - **Data leakage**: using test set during development invalidates results.

Overfitting signs: - Training loss decreases but validation loss increases. - Training accuracy high, validation accuracy much lower. - **Solutions**: dropout, data augmentation, early stopping, more data, L2 regularization, simpler model, transfer learning.

Data augmentation (artificially expand training set): - Random horizontal flip. - Random crop / resize. - Color jitter (brightness, contrast, saturation, hue). - Rotation, shear, perspective distortion. - Cutout / random erasing (hide random patches).

4.8a Classification Metrics: Accuracy, Precision, Recall, F1

Confusion Matrix:

```
              Predicted
            Pos    Neg
Actual Pos  TP     FN
       Neg  FP     TN
```

Definitions: - **Accuracy** = (TP + TN) / (TP + TN + FP + FN) — good for balanced datasets, misleading for imbalanced. - **Precision** = TP / (TP + FP) — of predicted positives, how many are correct? Important when false

positives are costly. - **Recall (Sensitivity)** = TP / (TP + FN) — of actual positives, how many did we catch? Important when false negatives are costly. - **F1 Score** = 2 · (Precision · Recall) / (Precision + Recall) — harmonic mean, balances precision and recall. Best for imbalanced datasets. - **Specificity** = TN / (TN + FP) — of actual negatives, how many did we correctly reject?

When to use which: - Balanced classes → Accuracy. - Imbalanced classes / medical screening → F1, Precision, Recall. - Spam detection → Precision (don't want false positives). - Disease detection → Recall (don't want false negatives).

4.8b Weight Initialization: Xavier & He

Why initialization matters: Bad initialization can cause vanishing or exploding gradients, especially in deep networks.

Xavier (Glorot) Initialization: - Designed for layers with **sigmoid or tanh** activations. - Samples from uniform or normal distribution with variance `Var(W) = 2 / (n_in + n_out)`. - Keeps forward and backward signal variance roughly constant.

He Initialization: - Designed for layers with **ReLU** activations. - Variance `Var(W) = 2 / n_in` (for ReLU). - Accounts for the fact that ReLU zeros out half the activations, so weights need larger initial variance.

PyTorch defaults: - Linear/Conv layers: typically Kaiming uniform (He) for ReLU, Xavier for sigmoid/tanh. - Proper initialization is especially critical for very deep networks (>30 layers).

4.8c Learning Rate Schedulers

Why needed: A constant learning rate may be too large late in training (causing oscillation) or too small early (slow convergence).

Common schedulers: 1. **StepLR**: reduce LR by factor `gamma` every `N` epochs. Simple and effective. 2. **ReduceLROnPlateau**: reduce LR when validation loss stops improving. Adaptive. 3. **Cosine Annealing**: LR follows cosine curve from initial to minimum value. Smooth decay, often used with warm restarts. 4. **Exponential Decay**: LR = $LR_0 \cdot \gamma^{epoch}$. Smooth continuous decay. 5. **OneCycleLR**: increase then decrease LR in one cycle. Often achieves faster convergence and better generalization.

Rule of thumb: - Start with LR finder (train for a few epochs with increasing LR, plot loss, pick LR just before loss shoots up). - Use scheduler to anneal LR as training progresses. - Combine with early stopping on validation loss.

4.9 Parameter Counting: Detailed Examples

Example 1: Conv layer with `11×11×30` input, five `3×3` filters, stride 2, no padding.

```
H_out = ⌊(11 − 3) / 2⌋ + 1 = 5
W_out = 5
C_out = 5
Output shape = 5 × 5 × 5
Params = (3·3·30 + 1) · 5 = 271 · 5 = 1355
```

Example 2: MLP with input 784, hidden 512, hidden 512, output 10.

```
Layer 1: 784·512 + 512 = 401,920
Layer 2: 512·512 + 512 = 262,656
Layer 3: 512·10 + 10 = 5,130
Total = 669,706
```

Example 3: Depthwise Separable Convolution. - Standard conv: 64 filters, 5×5, 128 channels → (5·5·128+1)·64 = 204,864. - Depthwise separable: - Depthwise: 128 filters of 5×5×1 → (5·5·1+1)·128 = 3,328. - Pointwise: 64 filters of 1×1×128 → (1·1·128+1)·64 = 8,256. - Total = 11,584. - **~18× fewer parameters**!

4.10 Classic Architectures: LeNet-5

- **1998**, Yann LeCun et al.
- 2 conv layers + 2 subsampling (pooling) + 2 fully connected.
- Activation: tanh/sigmoid (pre-ReLU era).
- Designed for **MNIST handwritten digit recognition** (28×28 grayscale).
- ~60K parameters.
- **Significance**: first successful CNN, proved the concept of learned features + weight sharing.

4.11 Classic Architectures: AlexNet

- **2012**, Krizhevsky, Sutskever, Hinton.
- 5 conv + 3 FC. **60M parameters**.
- **Key innovations that enabled deep CNNs**:
  1. **ReLU activation**: first successful use at scale. Avoided vanishing gradient of sigmoid/tanh. Faster convergence.
  2. **Dropout (0.5)**: in FC layers for regularization.
  3. **Local Response Normalization (LRN)**: lateral inhibition inspired by biological neurons. Later replaced by batch norm.
  4. **GPU training**: used 2 GTX 580 GPUs. Parallelized across GPUs.
  5. **Data augmentation**: random crops (256×256 from 224×224), horizontal flips, PCA color jittering.
- Won ImageNet 2012 by **10.8 percentage points** (26.2% → 15.3% error).
- **Triggered the deep learning revolution** in computer vision.

4.12 Classic Architectures: VGG

- **2014**, Simonyan & Zisserman.
- Very deep (16-19 layers, e.g., VGG-16, VGG-19).
- **Only 3×3 convolutions** throughout.
- **Why 3×3?**
  - Stack of three 3×3 has **same receptive field** as one 7×7: $3+2+2 = 7$ (each adds 2 to RF with stride 1).
  - **Fewer parameters**: $3\cdot(3\cdot3\cdot C^2) = 27C^2$ vs $7\cdot7\cdot C^2 = 49C^2$ → ~45% fewer.
  - **More non-linearities**: 3 ReLU activations instead of 1 → more expressive power, better discrimination.
- Very uniform architecture: conv → relu → conv → relu → pool → repeat.
- ~138M parameters (VGG-16) — mostly in FC layers.
- **No batch normalization in original** (added in later variants).
- **Insight**: depth matters. Small filters + more layers > large filters + fewer layers.

4.13 Classic Architectures: ResNet

- **2015**, He et al. “Deep Residual Learning for Image Recognition”.
- **Problem it solved**: vanishing gradients in very deep networks. Plain networks (VGG-like) degrade with depth beyond ~20 layers (not just overfitting — training error increases too).
- **Core idea: Residual learning**.
  - Instead of learning `H(x)` directly, learn `F(x) = H(x) − x`.
  - Output: `y = F(x) + x`.
  - If `F(x)` learns zero (easy), identity mapping `y = x` is preserved.
- **Skip connections (residual connections)**:
  - Provide a **shortcut path** for gradients.
  - `∂L/∂x = ∂L/∂F · ∂F/∂x + 1`. The `+1` guarantees gradients cannot vanish completely.
- Enables training of extremely deep networks: ResNet-50, ResNet-101, ResNet-152, ResNet-1000+.

Two types of blocks: 1. **Identity block**: `F(x) + x`. Used when input and output have same dimensions (same channels, same spatial size). 2. **Convolutional block**: `F(x) + W_s·x`. When dimensions change (e.g., after pooling, or changing number of channels), a **1×1 convolution** `W_s` projects the shortcut to match dimensions.

Bottleneck design (ResNet-50+): - Three layers per block: 1×1 (reduce channels) → 3×3 (spatial processing) → 1×1 (restore channels). - 1×1 conv reduces computation by decreasing channels before expensive 3×3. - Example: 256 channels → 1×1 to 64 → 3×3 → 1×1 to 256.

ResNet-50: ~25.6M parameters.

4.14 Transfer Learning Strategies

Why transfer learning works: - Early layers learn generic features (edges, textures) transferable across tasks. - Pre-trained models already have good feature representations. - Much less data needed, much faster convergence.

Feature extraction: - Freeze all pre-trained layers (backbone). - Replace final FC layer with new layer for target classes. - Train only the new head. - Best when: small dataset, similar to pre-training data (e.g., ImageNet).

Fine-tuning: - Unfreeze some or all layers. - Train entire network with **small learning rate** (e.g., 1e-5). - Best when: large dataset or very different from pre-training data.

Two-stage fine-tuning (recommended for best results): 1. Freeze backbone, train new head with moderate LR (e.g., 0.001) for several epochs. 2. Unfreeze backbone, train entire network with small LR (e.g., 1e-5) with weight decay. - **Why**: prevents large initial error of random head from corrupting good pre-trained features.

Replacing FC with SVM/RF: - Use CNN as fixed feature extractor up to final pooling layer. - Extract feature vectors for all images. - Train SVM or Random Forest on these features. - **Pros**: works with very small datasets; SVM may generalize better. - **Cons**: CNN features not fine-tuned for specific task; end-to-end training usually achieves better accuracy.

4.15 What CNN Layers Learn (Visualization)

- **First conv layers (layer 1-2)**: Gabor-like edge detectors, color blobs, oriented bars. These are **generic and transferable** across almost all vision tasks.
- **Second/Third conv layers (layer 3-5)**: textures, corners, simple patterns (grids, spots). Still fairly generic.
- **Deeper conv layers (layer 6+)**: object parts (wheel, eye, door, text). Task-specific but reusable.
- **Final FC layers**: combine presence/absence of object parts into **class scores**. Highly **task-specific**. Must be retrained for new datasets.

Transfer learning insight: freeze early layers (they're already good), retrain late layers (they need to adapt to new task).

---

5. OBJECT DETECTION AND SEGMENTATION WITH CNNs

5.1 Semantic Segmentation: FCN

Fully Convolutional Network (FCN): - **Key innovation**: replace all fully connected layers with 1×1 convolutions. - **Consequence**: network can accept **any input size** (not fixed like classification networks). Output size scales with input. - **Upsampling**: learned transposed convolutions or bilinear upsampling + convolution to recover spatial resolution. - **Skip connections**: combine high-resolution, low-semantic features from early layers with low-resolution, high-semantic features from late layers. - FCN-32s: upsample stride-32 features directly (coarse). - FCN-16s: add stride-16 features (better). - FCN-8s: add stride-8 features (finest). - **Limitation**: early FCNs struggled with fine boundaries and small objects.

5.2 Semantic Segmentation: U-Net

- **2015**, Ronneberger et al. for biomedical image segmentation.
- Now universally used for dense prediction tasks.

Architecture: - **Encoder (contracting path)**: repeated conv + max pool. - Captures global context (WHAT is present). - Reduces spatial resolution, increases channels. - Bottleneck has highest semantic information but lowest spatial resolution. - **Decoder (expanding path)**: upsampling + conv. - Recovers spatial resolution (WHERE it is). - **Skip connections**: concatenate high-resolution encoder features with decoder features at the **same resolution**. - Preserve fine-grained details (cell boundaries, edges, small structures) lost during pooling. - Without skip connections, output would be blurry.

Why U-Net works for biomedical images: 1. Skip connections preserve fine cellular boundaries critical for diagnosis. 2. Works well with **small datasets** (common in medicine) due to strong architectural inductive bias. 3. End-to-end trainable with pixel-wise loss (e.g., cross-entropy per pixel). 4. Can be seen as **image-to-image mapping**: input image → encoded representation → decoded segmented image.

Variants: U-Net++ (nested skip connections), Attention U-Net (adds attention gates to skip connections), V-Net (3D version).

5.3 Semantic Segmentation: DeepLab

- Uses **Atrous (Dilated) Convolution**:
  - Inserts holes (zeros) between kernel weights.
  - Increases receptive field without increasing kernel size or losing resolution.
  - Example: 3×3 dilated conv with rate 2 has RF of 5×5.
- **Atrous Spatial Pyramid Pooling (ASPP)**:
  - Parallel atrous convolutions with different rates (e.g., 6, 12, 18).
  - Captures multi-scale context.
  - Objects at different scales are handled simultaneously.
- **DeepLabv3+**: adds decoder module to refine boundaries, combining encoder features with low-level features.

5.4 Object Detection: R-CNN (2014)

Two-stage detector paradigm: 1. Generate region proposals. 2. Classify each region.

R-CNN pipeline: 1. **Selective Search**: generates ~2000 region proposals per image based on color, texture, size, and fill similarity. 2. Each proposal cropped and warped to fixed size (e.g., 227×227). 3. Each warped region fed through CNN **independently** (~2000 forward passes). 4. CNN features classified by **SVM** (one per class). 5. **Bounding box regression** refines proposal coordinates.

Problems: - **Very slow**: ~2000 forward passes per image (~50 seconds on GPU). - Multi-stage pipeline: CNN → SVM → regression. Not end-to-end trainable. - Warping distorts aspect ratios.

5.5 Object Detection: Fast R-CNN (2015)

Key insight: share convolution across proposals.

Pipeline: 1. Image goes through CNN **once** → shared convolutional feature map. 2. Region proposals (from Selective Search) projected onto the feature map using their coordinates. 3. **RoI Pooling**: extracts fixed-size feature vector for each proposal from shared feature map. - Divide proposal region into fixed grid (e.g., 7×7). - Max pool within each cell → fixed-size output regardless of input proposal size. 4. FC layers classify and perform bounding box regression.

Advantages: - **Much faster** (~2 seconds vs ~50 seconds) because convolution is shared. - **Multi-task loss**: classification + regression trained jointly (end-to-end). - Still uses external Selective Search.

RoI Pooling problem: quantizes proposal coordinates to integer feature map coordinates. Causes **misalignment** between proposal and features.

5.6 Object Detection: Faster R-CNN (2015)

Key innovation: replace external Selective Search with **Region Proposal Network (RPN)** inside the network.

Pipeline: 1. Image through CNN → shared feature map. 2. **RPN**: slides a small network over shared feature map. - At each location, evaluates a set of **anchor boxes** (predefined sizes/aspect ratios, e.g., 3 scales × 3 ratios = 9 anchors per location). - For each anchor, predicts: - **Objectness score**: is this anchor an object or background? - **Bounding box refinement**: 4 offsets (dx, dy, dw, dh) to refine anchor coordinates. 3. Top-N proposals by objectness score fed to Fast R-CNN detector. 4. Fast R-CNN head classifies proposals and refines boxes.

Advantages: - **Unified, end-to-end trainable** (alternating or joint training). - **~10× faster** than R-CNN (~200ms per image). - Near real-time performance. - Shared features between RPN and detector → efficient.

5.7 Object Detection: YOLO (You Only Look Once)

One-stage detector paradigm: - No explicit region proposal step. - Divides image into grid. Each cell predicts boxes and classes **simultaneously**. - **Single forward pass** → extremely fast (45 FPS for YOLOv1).

YOLOv1 architecture: - Image divided into `S×S` grid (e.g., 7×7). - Each grid cell predicts: - `B` bounding boxes (each with x, y, w, h, confidence). - `C` class probability scores (conditional on cell containing object).

Output:

$$\text{Total outputs} = S^2 \times (B \cdot 5 + C)$$

where 5 = `(x, y, w, h, confidence)`.

Example: YOLOv1 on Pascal VOC (`S=7, B=2, C=20`):

`7×7×(2×5 + 20) = 49×30 = 1470 outputs per image`

Confidence score: `P(object) × IoU(pred, truth)`. - During inference, boxes with low confidence are filtered out before NMS.

Loss function (multi-part): - Coordinate loss (MSE for x, y, w, h) — only for boxes responsible for ground truth. - Confidence loss (MSE for objectness). - Classification loss (cross-entropy for class probabilities) — only for cells containing objects. - Different weights balance localization vs classification.

Why "You Only Look Once": - R-CNN family: first proposes regions, then classifies each → two looks. - YOLO: predicts boxes and classes simultaneously in one network evaluation → one look.

Challenges of early YOLO: - Coarse grid (7×7) → struggles with small objects and dense scenes. - Each cell predicts only 2 boxes → limited for multiple small objects in same cell. - Struggles with unusual aspect ratios.

Later versions: - YOLOv2: batch norm, anchor boxes, multi-scale training. - YOLOv3: multi-scale predictions at 3 scales, better backbone (Darknet-53). - YOLOv4/v5/v8: improved backbones, augmentation, anchor-free designs, better loss functions.

5.8 DETR (Detection Transformer)

- **2020**, Carion et al. "End-to-End Object Detection with Transformers."
- Represents a paradigm shift from anchor-based detection to **set-based detection** using Transformers.

Motivation: - R-CNN and YOLO rely on hand-designed components: anchor boxes, NMS, RoI pooling. - DETR simplifies object detection into an **end-to-end set prediction problem** using the Transformer encoder-decoder architecture. - No anchors, no NMS, no region proposals.

Architecture: 1. **CNN backbone** (e.g., ResNet-50): extracts compact feature representation from the image. 2. **Encoder**: 1×1 convolution reduces channel dimensions, then standard Transformer encoder with self-attention. - Self-attention reasons globally over the entire image, encoding relationships between all spatial locations. 3. **Decoder**: standard Transformer decoder but with **learned object queries** (instead of previous tokens). - Each object query is a learned embedding vector representing a "slot" to be filled with one detected object. - Number of queries is fixed and larger than max objects per image (typically 100 queries). 4. **Prediction heads** (FFN): for each query, predicts: - **Class label** (including a special "no object" class ∅). - **Bounding box** (normalized center coordinates + width/height).

Set-based loss: Hungarian matching: - DETR produces a fixed set of predictions (e.g., 100 boxes). Ground truth may have fewer objects (e.g., 3). - Need to find the **optimal bipartite matching** between predictions and ground truth. - Uses the **Hungarian algorithm** (assignment problem) to match each ground-truth object to its best prediction. - Cost = classification error + L1 box distance + generalized IoU (gIoU) loss. - Unmatched predictions are penalized as "no object" (∅).

Key advantages: 1. **True end-to-end**: no anchors, no NMS, no region proposals. Direct set prediction. 2. **Global reasoning**: encoder self-attention reasons about entire image context, unlike local receptive fields in CNN detectors. 3. **Simpler pipeline**: fewer hyperparameters (no anchor sizes, aspect ratios, NMS thresholds).

Key limitations: 1. **Slow convergence**: requires much longer training than Faster R-CNN (500 epochs vs ~12). 2. **Struggles with small objects**: coarse feature maps from backbone limit small object detection. 3. **Fixed number of queries**: must set maximum number of detections upfront. 4. **Quadratic complexity**: self-attention is $O(N^2)$ where N = spatial resolution × queries.

Improvements: - **Deformable DETR**: uses deformable attention (sparse sampling) to reduce complexity and improve small object detection. - **DAB-DETR**: dynamic anchor boxes as queries, better convergence. - **DN-DETR**: denoising training stabilizes learning.

Comparison with two-stage and one-stage detectors:

Property	Faster R-CNN	YOLO	DETR
Design	Two-stage	One-stage	Set-based Transformer
Anchors	Yes (anchors + RPN)	Yes (grid + anchors)	**No**
NMS	Required	Required	**Not needed**
Global context	Limited (local receptive fields)	Limited (local grid)	**Full self-attention**
Training speed	Moderate	Fast	**Slow (500+ epochs)**
Small objects	Good (RPN multi-scale)	Moderate	**Poor** (coarse features)
Simplicity	Complex (many components)	Moderate	**Simple (fewer hyperparams)**

5.9 Instance Segmentation: Mask R-CNN

- Extends Faster R-CNN with a **third branch**: mask prediction.

Pipeline: 1. Same as Faster R-CNN: RPN proposals + RoI classification/regression. 2. **Mask branch**: for each detected object, predicts a **binary segmentation mask** (e.g., 28×28) in parallel with class and box.

Key innovation: RoIAlign: - **RoI Pooling** (Fast/Faster R-CNN): quantizes RoI boundaries and bins to integer coordinates → misalignment. Acceptable for coarse classification but problematic for pixel-accurate masks. - **RoIAlign**: uses **bilinear interpolation** to sample features at continuous locations **without quantization**. - Divide RoI into equal bins. - Sample 4 points per bin using bilinear interpolation. - Max/Average pool the samples. - **Preserves spatial precision** → essential for mask prediction.

Advantages: - Detects and segments each instance separately. - Mask branch adds minimal overhead (~10-15% slower than Faster R-CNN). - State-of-the-art for instance segmentation.

For blood cells: Mask R-CNN separates touching cells by predicting individual masks for each instance. Semantic segmentation (U-Net alone) would merge them.

5.10 Evaluation Metrics for Detection & Segmentation

IoU (Intersection over Union):

```
IoU = Area(Intersection) / Area(Union)
```

- Measures overlap between predicted and ground-truth boxes/masks.
- `IoU > 0.5`: standard threshold for “correct” detection.
- `IoU > 0.75`: stricter threshold.
- COCO uses `IoU = 0.5:0.95` (averaged across thresholds).

Precision and Recall: - `Precision = TP / (TP + FP)` — of all detections, how many are correct? - `Recall = TP / (TP + FN)` — of all ground-truth objects, how many did we find?

Average Precision (AP): - Area under the Precision-Recall curve for one class. - Computed by varying confidence threshold.

mAP (mean Average Precision): - Mean of AP over all classes. - `mAP@0.5`: averaged at IoU=0.5 (Pascal VOC metric). - `mAP@0.5:0.95`: averaged over IoU thresholds 0.5, 0.55, …, 0.95 (COCO metric, much stricter).

Non-Maximum Suppression (NMS): - **Problem**: detectors output multiple overlapping boxes for same object. - **Algorithm**: 1. Sort all boxes by confidence score (descending). 2. Select highest-confidence box, add to final detections. 3. Remove all boxes overlapping selected box with `IoU > threshold` (e.g., 0.5). 4. Repeat until no boxes remain. - **Variants**: Soft-NMS (decreases scores instead of hard removal), DIoU-NMS (considers distance between centers).

5.11 Anchor Boxes & Objectness

Anchor boxes: - Pre-defined bounding boxes of various scales and aspect ratios placed at each spatial location. - Detector predicts **offsets** relative to anchors, not absolute coordinates. - Makes learning easier because offsets are usually small numbers. - Common aspect ratios: 1:1, 1:2, 2:1. Common scales: small, medium, large.

Objectness score: - Probability that a region contains an object (vs background). - In YOLO: `P(object) × IoU(pred, truth)`. - In RPN: binary classification (object vs not-object). - Used to filter out background regions before classification.

---

6. GENERATIVE ADVERSARIAL NETWORKS

6.1 GAN Fundamentals

Components: - **Generator** `G(z)`: takes random noise `z` (typically from simple distribution like Gaussian) and generates realistic data `G(z)` that looks like training data. - **Discriminator** `D(x)`: binary classifier that distinguishes real data from generated (fake) data.

Objective (Minimax Game):

```
min_G max_D V(D, G) = E_{x~pdata}[log D(x)] + E_{z~pz}[log(1 − D(G(z)))]
```

Discriminator's goal: - Maximize `log D(x)` for real data (want D(x) → 1). - Maximize `log(1 − D(G(z)))` for fake data (want D(G(z)) → 0). - Equivalently: maximize probability of correctly classifying real as real AND fake as fake.

Generator's goal: - Minimize `log(1 − D(G(z)))` — wants D(G(z)) → 1 (fool discriminator). - In practice, often maximize `log(D(G(z)))` instead — same fixed point but stronger gradients early in training.

Nash Equilibrium: - At theoretical equilibrium: `D(x) = 0.5` for all `x` (cannot distinguish real from fake). - Generator has perfectly learned the data distribution `p_data`. - In practice, equilibrium may never be reached.

6.2 GAN Training Challenges & Instability

1. Mode Collapse: - Generator learns to produce a **limited variety** of outputs (or even a single output) that fools the discriminator. - Example: GAN trained on handwritten digits always generates the same digit "3" because it's good enough to fool D. - **Solutions**: - WGAN (Wasserstein distance avoids mode collapse). - Minibatch discrimination (D evaluates diversity within a batch). - Unrolled GANs (D looks ahead to G's updates). - Add noise to inputs.

2. Vanishing Gradients: - If D becomes too good too quickly, $D(G(z)) \to 0$. - Gradient for G: $\partial/\partial G \log(1-D(G(z))) = -1/(1-D) \cdot \partial D/\partial G$. When D≈0, gradient ≈ −1 → vanishes. - **Solution**: use alternative objective $\max_G \log(D(G(z)))$ (non-saturating loss).

3. Non-Convergence: - G and D are in an adversarial game. No single loss minimum guarantees convergence. - Loss curves oscillate rather than monotonically decrease. - Hard to tell if training is "working" from losses alone.

4. Balancing Act: - If D is too weak → G doesn't learn meaningful features (already fools D). - If D is too strong → G cannot improve (vanishing gradients). - Often need different learning rates or update frequencies (e.g., train D 5 steps per G step).

5. No Clear Evaluation Metric: - Unlike classification where accuracy is clear, GAN quality is subjective. - Metrics: Inception Score (IS), Frechet Inception Distance (FID). Lower FID = better.

6.3 Wasserstein GAN (WGAN)

Problem with standard GAN: JS divergence between distributions can be constant (zero gradient) when distributions don't overlap, causing vanishing gradients.

WGAN solution: - Replace JS divergence with **Earth Mover's (Wasserstein) distance**. - Wasserstein distance is smooth and provides gradients even when distributions don't overlap. - Discriminator becomes a **critic** that estimates Wasserstein distance (must be Lipschitz continuous). - Enforce Lipschitz constraint via: - Weight clipping (original WGAN). - Gradient penalty (WGAN-GP, more stable).

Benefits: - More stable training. - Meaningful loss curve (correlates with sample quality). - No mode collapse issues.

6.4 Conditional GAN (cGAN)

- Both G and D receive auxiliary conditioning information y.
- $G(z, y)$ generates samples conditioned on y.
- $D(x, y)$ discriminates conditioned on y.

Conditioning can be: - Class label (generate image of digit "7"). - Text description (generate image of "red bird on branch"). - Another image (image-to-image translation).

Applications: - Class-conditioned image generation. - Image-to-image translation (pix2pix). - Super-resolution (SRGAN).

6.5 CycleGAN: Unpaired Image-to-Image Translation

Problem: pix2pix requires **paired training data** (aligned input-output pairs), which is expensive to collect.

CycleGAN solution: learns mapping between two domains $X \leftrightarrow Y$ **without paired examples**.

Architecture: - Two generators: $G: X \to Y$ and $F: Y \to X$. - Two discriminators: D_Y (distinguishes real Y from G(X)) and D_X (distinguishes real X from F(Y)).

Losses: 1. **Adversarial loss**: standard GAN losses for both directions. 2. **Cycle consistency loss**: $L_{cycle} = E_x[||F(G(x)) - x||_1] + E_y[||G(F(y)) - y||_1]$ - Forward cycle: $x \to G(x) \to F(G(x)) \approx x$. - Backward cycle: $y \to F(y) \to G(F(y)) \approx y$.

Why cycle consistency is CRITICAL: - Without it, G could learn an arbitrary mapping that ignores input content. - Example: map every horse photo to the same zebra photo regardless of pose. - Cycle loss forces the translated image to retain enough information to reconstruct the original. - Ensures **semantic content is preserved** across domains.

Identity loss (optional): - `G(x) ≈ x` when `x` is already in domain Y. - Prevents unnecessary modifications when input already matches target domain.

Applications: horses ↔ zebras, summer ↔ winter, Monet ↔ photo, sketches ↔ photos.

6.6 GAN Applications

1. **Image generation**: faces (StyleGAN), art, scenes.
2. **Image-to-image translation**: domain transfer (CycleGAN, pix2pix).
3. **Super-resolution**: generate realistic high-frequency details (SRGAN, ESRGAN).
4. **Data augmentation / class imbalance**: generate synthetic minority-class samples.
    - Train GAN on rare class → generate realistic samples → augment training set.
    - Especially useful in medical imaging (rare diseases, tumors).
5. **Inpainting**: fill missing regions realistically.
6. **Style transfer**: apply artistic style to content images.
7. **Text-to-image synthesis**: DALL-E, Stable Diffusion (diffusion-based, GAN-inspired).

6.7 Evaluating GANs: IS and FID

Inception Score (IS): - Uses pre-trained Inception network to classify generated images. - High IS: generated images are diverse (high entropy over classes) and each image is confidently classified (low conditional entropy). - **Limitations**: insensitive to mode collapse within classes, doesn't compare to real data.

Frechet Inception Distance (FID): - Computes mean and covariance of Inception features for real and generated images. - `FID = ||μ_real − μ_gen||² + Tr(Σ_real + Σ_gen − 2√(Σ_real·Σ_gen))`. - **Lower FID = better quality and diversity**. - **Preferred metric**: directly compares generated distribution to real distribution. - Sensitive to mode collapse and sample quality.

---

7. ATTENTION MODELS AND VISION TRANSFORMERS

7.1 The Limitations of CNNs for Long-Range Dependencies

- CNN receptive field grows linearly with depth.
- Early layers only see local neighborhoods.
- Capturing long-range relationships (e.g., a person's face and feet) requires very deep networks or dilated convolutions.
- **Attention mechanism**: directly models relationships between any two positions, regardless of distance.

7.2 Self-Attention Mechanism

Core idea: for each position in a sequence, compute a weighted sum of all positions, where weights depend on content similarity.

Mathematical formulation: For input sequence `X` (each row is a token embedding):

```
Q = X · W_Q    (Queries)
K = X · W_K    (Keys)
V = X · W_V    (Values)

Attention(Q, K, V) = softmax( (Q · K^T) / √d_k ) · V
```

Dimensions: - `Q, K, V`: each has shape `(sequence_length, d_k)` or `(sequence_length, d_v)`. - `W_Q, W_K, W_V`: learned projection matrices.

Intuition: - **Query** (from token i): “What am I looking for?” - **Key** (from token j): “What do I contain?” - **Value** (from token j): “What information do I provide?” - **Attention weight** between i and j: `softmax(Q_i · K_j / √d_k)`. - High similarity between Query and Key → strong attention. - **Output for token i**: weighted sum of all Values, where weights are attention scores.

Scaling by √d_k: - For large `d_k`, dot products `Q·K^T` grow in magnitude. - Large values push softmax into regions with extremely small gradients (saturation). - Dividing by √d_k keeps dot products in a reasonable range (~O(1)). - Prevents vanishing gradients during backpropagation.

7.3 Multi-Head Self-Attention

- Runs the attention mechanism **multiple times in parallel** with different learned projections.
- Each “head” can attend to different representation subspaces.
  - Example: one head attends to syntax, another to semantics, another to spatial relationships.
- Outputs from all heads are concatenated and linearly projected.

```
MultiHead(Q, K, V) = Concat(head_1, ..., head_h) · W_O
where head_i = Attention(Q·W_Qi, K·W_Ki, V·W_Vi)
```

- **Benefit**: multiple attention patterns learned simultaneously → richer representations.

7.4 Transformer Encoder Block

Each encoder layer consists of: 1. **Multi-Head Self-Attention**. 2. **Add & Norm**: residual connection + Layer Normalization. 3. **Feed-Forward Network (FFN)**: two linear layers with non-linearity (ReLU/GELU) applied to each position independently. - `FFN(x) = max(0, x·W_1 + b_1)·W_2 + b_2` - Inner dimension is typically larger than model dimension (e.g., 4×). 4. **Add & Norm**: residual connection + Layer Normalization.

Layer Normalization (not Batch Normalization): - Normalizes across the feature dimension for **each sample independently**. - `LN(x) = γ · (x − μ) / (σ + ε) + β` - Unlike Batch Norm, doesn’t depend on batch statistics → suitable for variable batch sizes and sequence models.

Stacking: Transformer encoder is a stack of N such layers (e.g., N=12 for BERT-base, ViT-Base).

7.5 Vision Transformer (ViT)

Motivation: apply standard Transformer (from NLP) directly to images.

Pipeline: 1. **Patch embedding**: Split image into non-overlapping patches (e.g., 16×16 for 224×224 image → 14×14 = 196 patches). - Flatten each patch and linearly project to embedding dimension D (e.g., 768). - Each patch becomes a “token” analogous to a word token in NLP. 2. **Position embeddings**: Add learned or sinusoidal position vectors to each patch token. - **Critical**: self-attention is permutation-invariant. Without position embeddings, shuffling patches would produce the same output. - Encode spatial information. 3. **Class token**:

Prepend a learnable `[CLS]` token to the sequence. - Its final representation aggregates global information via self-attention. - Used for classification (mirrors BERT's `[CLS]` design). 4. **Transformer encoder**: Pass through standard Transformer encoder stack (multi-head attention + FFN + LayerNorm + residuals). 5. **MLP head**: Final `[CLS]` token output fed to MLP classifier head.

Comparison with CNNs:

Property	CNN	ViT
Inductive bias	Locality, translation invariance (built-in via convolutions)	Minimal (only position embeddings). Must learn spatial relationships from scratch.
Receptive field	Grows with depth. Early layers see only local neighborhoods.	**Global from first layer**. Any token can attend to any other.
Computation	Efficient (O(N) with weight sharing, local operations).	**Quadratic in sequence length** $O(N^2)$ for self-attention.
Data efficiency	Good with small datasets due to strong inductive bias.	**Needs large pre-training datasets** (e.g., JFT-300M) to learn spatial patterns.
Interpretability	Feature maps show what each layer detects.	Attention maps show which patches interact.

Why ViT needs large pre-training: - CNNs have strong built-in assumptions (locality, translation invariance, hierarchical composition). - Transformers have minimal inductive bias. They must learn everything from data. - On ImageNet-1k alone, ViT underperforms ResNet. With JFT-300M pre-training, ViT matches or exceeds CNNs.

7.6 Why Transformers Excel at Multi-Modal Tasks

Image + Text fusion: - In CNNs: images are 2D grids, text is 1D sequences. Fusing them requires specially designed adapters (e.g., BERT+CNN hybrids, attention over CNN features). - In Transformers: **both modalities become sequences of tokens**. - Images → patch tokens. - Text → word/subword tokens. - **Self-attention naturally learns cross-modal relationships**. - Example: "cat" text token attends strongly to cat-shaped image patches. - Just concatenate sequences and add **modality-type embeddings** (learned vectors indicating whether token is image or text). - No special fusion architecture needed.

Models using this approach: - **CLIP** (Contrastive Language-Image Pre-training): learns joint embedding space for images and text → zero-shot classification. - **DALL-E / Stable Diffusion**: text-to-image generation. - **GPT-4V**: vision-language understanding.

7.7 Attention Maps & Interpretability

- **Attention maps**: visualize which patches/tokens the model focuses on.
- If a Vision Transformer shows strong attention from a cat's eye patch to distant patches (ears, whiskers) but weak attention to nearby background:
  - The model learned **semantic grouping** and **object coherence**.
  - It recognizes that eye, ears, and whiskers are parts of the same semantic object.
  - This demonstrates that attention captures **long-range semantic dependencies**, not just spatial proximity.
- **Dynamic**: attention patterns change per image. CNN kernels are fixed after training.

- **Useful for debugging**: if model attends to background instead of object, it may be learning spurious correlations.

7.8 Computational Challenges of Self-Attention

Quadratic complexity: `O(N²)` where `N` = number of tokens.

Example: - Image: 1024×1024. Patch size: 16×16. - `N = (1024×1024) / (16×16) = 4096` tokens. - Attention matrix: 4096 × 4096 = ~16.7M entries. - Much heavier than CNNs for high-resolution images.

Solutions for efficiency: 1. **Sparse attention**: only attend to nearby patches (local attention) + a few global tokens. 2. **Windowed attention** (Swin Transformer): compute self-attention within non-overlapping windows, then shift windows across layers. 3. **Linear attention approximations**: kernel methods to approximate softmax attention in O(N) time. 4. **Hierarchical architectures**: process high-resolution with CNN early, then Transformer later. 5. **Token pruning / merging**: dynamically remove redundant tokens.

7.9 Hybrid CNN-Transformer Models

Problem: pure ViT is inefficient for high-resolution images and needs huge pre-training data.

Solution: combine CNN stem with Transformer body.

Examples: - **CoAtNet**: convolutional stem + Transformer blocks. CNN provides locality and efficiency; Transformer provides global reasoning. - **Swin Transformer**: hierarchical vision Transformer using shifted windows. Maintains multi-scale feature maps like CNNs but with self-attention. - **ConvNeXt**: modernized pure CNN that incorporates Transformer design choices (larger kernels, LayerNorm, GELU) → nearly matches Transformer performance without attention.

Future direction: hybrid models may dominate — CNNs for efficiency and inductive bias, Transformers for global relationships and multi-modal fusion.

PART 2 - MASSIVE QUESTION BANK (323+ Q&A)

SECTION C: SEGMENTATION AND MOTION (32 Questions)

C1. Identify an example where a single global threshold is unfeasible for segmentation. A: A document scanned under uneven lighting (one side in shadow). Text on the bright side may be brighter than background on the dark side. A single T cannot separate both simultaneously. Solution: adaptive thresholding or illumination normalization.

C2. How does Otsu's method choose the optimal threshold? A: It maximizes between-class variance: sigma^2_b(T) = omega0(T)*omega1(T)*[mu0(T)-mu1(T)]^2, where omega are class probabilities and mu are class means. Equivalent to minimizing within-class variance.

C3. When does Otsu's method fail? A: When the histogram is unimodal, modes overlap heavily, or illumination varies spatially. It assumes a roughly bimodal histogram.

C4. Compare adaptive thresholding with Otsu thresholding. A: Otsu: global, automatic, fast, optimal for bimodal histograms with uniform illumination. Adaptive: local, computes T per pixel from neighborhood, better for uneven lighting/shadows, slower, requires blockSize and C.

C5. What is the difference between ADAPTIVE_THRESH_MEAN_C and ADAPTIVE_THRESH_GAUSSIAN_C? A: MEAN_C: threshold = mean(neighborhood) - C. GAUSSIAN_C: threshold = Gaussian-weighted mean of neighborhood - C. Gaussian gives more weight to center pixels, producing smoother thresholds.

C6. Why does K-Means segmentation sometimes produce fragmented regions? A: K-Means clusters pixels purely by color in feature space (e.g., RGB) with no spatial coherence term. Two adjacent pixels with slightly different colors may fall into different clusters. Solutions: spatial K-means (add x,y coordinates), mean shift, or graph-based methods.

C7. What are the four mask labels in GrabCut, and why are both 'probable' categories needed? A: Definite background, probable background, definite foreground, probable foreground. The probable categories allow the iterative GMM+graph-cut algorithm to refine its decisions during optimization. Definite pixels are fixed constraints. This flexibility is essential because the initial rectangle is imprecise.

C8. Why is GrabCut considered semi-automatic? A: Because it requires user input (a rectangle or rough mask) to initialize the foreground/background separation. After initialization, it iteratively refines using GMMs and graph cuts automatically.

C9. Why do traditional computer vision methods often perform worse than DL-based methods in semantic segmentation? A: 1) Hand-crafted features don't generalize to object variability. 2) No global context - pixel decisions are purely local. 3) Cannot learn or improve with more data. 4) DL methods learn hierarchical features, combine global context with fine detail via skip connections, and improve with data.

C10. What is the difference between semantic and instance segmentation? A: Semantic: classify every pixel into a category (all people share same label). Instance: classify pixels AND distinguish individual objects (person 1, person 2). Semantic uses FCN/U-Net/DeepLab. Instance uses Mask R-CNN/DETR.

C11. What is panoptic segmentation? A: It combines both semantic segmentation (for background/stuff classes like sky, road) and instance segmentation (for countable things like people, cars) into a unified output.

C12. Comment: 'The optical flow is not always a correct estimate of the motion.' Provide two examples. A: 1) A rotating uniform white sphere under static lighting: no brightness change -> zero optical flow despite rotation. 2) A moving cloud shadow across a parking lot: ground is static but optical flow detects motion due to changing illumination.

C13. What is the brightness constancy assumption, and when does it fail? A: $I(x,y,t) = I(x+dx,y+dy,t+dt)$. It assumes pixel brightness doesn't change between frames. Fails under: changing illumination, moving shadows,

specular reflections, translucent objects, and when brightness changes without motion.

C14. What is the aperture problem, and how does Lucas-Kanade handle it? A: The aperture problem is the ambiguity of motion along an edge in a small window. Lucas-Kanade solves for flow using a least-squares system over a local window. The system requires the matrix [[sum Ix^2, sum Ix*Iy],[sum Ix*Iy, sum Iy^2]] to be invertible, which requires textured regions (both eigenvalues large). On edges, the matrix is singular.

C15. Why does Lucas-Kanade with a pyramid (calcOpticalFlowPyrLK) handle larger motions? A: It computes flow from coarse to fine pyramid levels. The flow estimate from a coarser level warps the finer level, reducing the displacement magnitude at each level so the small-motion assumption holds.

C16. For real-time human tracking, would you use dense or sparse optical flow? Why? A: Sparse optical flow (Lucas-Kanade with goodFeaturesToTrack). Dense optical flow computes flow for every pixel and is computationally expensive. Sparse flow tracks only salient corners, which is sufficient for tracking and much faster.

C17. How would you segment touching blood cells in a microscopy image? A: 1) U-Net or adaptive thresholding for binary mask. 2) Distance transform on binary mask. 3) Find local maxima (cell centers). 4) Watershed using maxima as markers to split touching cells. Alternatively: Mask R-CNN for end-to-end instance segmentation.

C18. What is Mean Shift segmentation, and what is its advantage over K-Means? A: Mean Shift is a mode-seeking clustering algorithm that finds dense regions in feature space without specifying k. Advantage: no need to choose k; more robust to outliers. Disadvantage: slower than K-Means.

C19. What pre-processing step often improves thresholding results? A: Applying a mild Gaussian blur before thresholding reduces noise and improves stability, especially for adaptive thresholding.

C20. True or False: Optical flow always provides the true physical motion of objects in a scene. A: False. Optical flow measures apparent motion of brightness patterns. It fails for: brightness changes without motion, no texture (uniform surfaces), aperture problem, and occlusions.

C21. What is the role of the distance transform in watershed-based cell segmentation? A: The distance transform computes the distance from each foreground pixel to the nearest background pixel. Local maxima in this transform correspond to cell centers, which serve as markers for the watershed algorithm to split touching cells at ridge lines.

C22. Why is U-Net preferred over a simple encoder-decoder for biomedical image segmentation? A: Skip connections concatenate high-resolution encoder features with decoder features at the same resolution. This preserves fine details like cell boundaries and small structures that are lost during pooling in the encoder. Essential for precise medical analysis.

C23. What is graph-cut optimization in the context of segmentation? A: The image is modeled as a graph where pixels are nodes and edges represent similarity/penalties. Segmentation becomes a minimum-cut problem

that separates foreground from background. GrabCut uses iterated graph cuts with GMMs to refine the segmentation energy.

C24. What is the main computational bottleneck of dense optical flow methods like Farneback? A: They compute flow for every pixel, requiring expensive operations (polynomial expansion, variational optimization) across the entire image. This makes them unsuitable for real-time applications on high-resolution video.

C25. How does morphological opening help in segmentation post-processing? A: Opening = erosion followed by dilation. It removes small noise objects and thin protrusions while preserving the approximate size and shape of larger objects. Useful after thresholding to clean up binary masks.

C26. Why might K-Means with k=3 fail to segment an image with 4 dominant colors? A: K-Means partitions data into exactly k clusters. With k=3, two dominant colors would be forced into the same cluster, merging distinct regions. The algorithm also converges to local minima, so initialization matters.

C27. What is the relationship between the brightness constancy equation and the Lucas-Kanade method? A: The brightness constancy equation Ix*u* + *Iy*v + It = 0 is under-constrained (one equation, two unknowns). Lucas-Kanade assumes the flow (u,v) is constant in a local window, yielding an over-determined system solvable by least squares.

C28. What would happen if you applied global thresholding to an MRI with bias field? A: The bias field causes smooth intensity variation across the image. A global threshold would fail to separate tissue consistently across the image because the same tissue type has different intensities in different regions. Adaptive methods or bias field correction are needed.

C29. In GrabCut, what happens if the user initializes with a very tight rectangle around the object? A: A tight rectangle provides strong definite foreground constraints but may cut off object parts (e.g., hair, thin structures). The algorithm has less freedom to refine boundaries. A slightly looser rectangle is often better to let the GMM learn the object appearance.

C30. What is the difference between ‘stuff’ and ‘things’ in segmentation terminology? A: ‘Things’ are countable objects (people, cars, animals) that instance segmentation separates individually. ‘Stuff’ is amorphous background material (sky, grass, road) that semantic segmentation labels uniformly without instances.

C31. For each of the following optical flow fields, describe a scene and a camera motion that could produce it: - (a) All vectors point outward radially from center - (b) All vectors point in the same direction with similar magnitude A: - (a) **Zooming in / camera moving forward** toward a static scene (optic expansion). All points appear to move away from the focus of expansion (typically image center if moving straight forward). Scene: hallway, road, or tunnel. Camera: moving forward along optical axis.

- b. **Camera translating sideways (pure translation)** with no rotation. All background points move in parallel (opposite to camera motion) with similar magnitude (if scene is roughly planar and perpendicular to view). Scene: looking out side window of moving car. Camera: pure lateral translation.

C32. The optical flow is not always a correct estimate of motion. Give a scene and camera configuration where OF fails to capture true motion. A: Example 1: A rotating uniform white sphere under static lighting. The sphere's surface brightness doesn't change as it rotates (uniform albedo), so optical flow is zero despite physical rotation.

Example 2: A static scene with a moving shadow (e.g., cloud shadow moving across a parking lot). The ground is stationary but brightness changes, so optical flow detects non-zero motion despite no physical movement.

Example 3: Aperture problem: a camera translating parallel to a straight edge. The edge's motion perpendicular to itself is visible, but motion along the edge is invisible to a local window → incomplete flow.

D1. In an AlexNet-like architecture, what is learned in the first conv layers vs last FC layers? A: First conv layers: low-level generic features (edges, corners, color blobs, textures). These are transferable. Last FC layers: task-specific high-level reasoning combining object parts into class scores. Must be retrained for new tasks.

D2. Comment: 'It is possible to replace the fully connected layers of a CNN with an SVM classifier.' A: True. This is a valid transfer learning approach. Use the CNN as a feature extractor up to the final pooling layer, extract feature vectors, and train an SVM. Pros: works with small data. Cons: CNN features are not fine-tuned for the new task, which end-to-end training would achieve.

D3. Why are CNNs better than fully connected networks (MLPs) for image classification? A: 1) Sparse connectivity: local patches only. 2) Weight sharing: same filter everywhere -> fewer parameters and translation invariance. 3) Hierarchical features: edge -> texture -> part -> object. 4) Preserves spatial structure.

D4. What is the vanishing gradient problem, and how do ResNet and ReLU address it? A: In deep networks, repeated multiplication of small gradients (<1) causes gradients to shrink exponentially. ReLU: gradient is 1 for positive inputs. ResNet: skip connections provide direct gradient path: dL/dx = dL/dF * dF/dx + 1. The +1 guarantees gradients cannot vanish completely.

D5. What is the difference between max pooling and average pooling? A: Max pooling takes the maximum in a window, selecting the most salient feature and providing translation invariance. Average pooling takes the mean, which is smoother and preserves more background information. Max is standard for classification; average/GAP replaces FC layers.

D6. Why is a stack of three 3x3 convolutions better than a single 7x7 convolution? A: 1) Fewer parameters: 27*C^2 vs 49*C^2. 2) More non-linearities: three ReLUs instead of one -> more expressive power. 3) Same receptive field: both cover 7x7 region.

D7. What is the purpose of dropout, and why is it only applied during training? A: Dropout randomly deactivates neurons during training to prevent co-adaptation and reduce overfitting. During inference, all neurons must be active to use the full learned model. Applying dropout at test time would produce stochastic, inconsistent predictions.

D8. Explain the difference between model.train() and model.eval() in PyTorch. A: model.train() enables training-specific behaviors: dropout is active, batch normalization uses batch statistics. model.eval() disables dropout and makes batch norm use running mean/variance. Using eval() during inference is mandatory for deterministic predictions.

D9. What is batch normalization, and how does it help training? A: BN normalizes activations per mini-batch to zero mean and unit variance, then learns scale gamma and shift beta. Benefits: reduces internal covariate shift, allows higher learning rates, reduces initialization dependence, acts as a regularizer.

D10. What is data augmentation, and why is it useful? Give three examples. A: Data augmentation artificially expands the training set by applying random transformations. It reduces overfitting by exposing the model to more variability. Examples: random horizontal flip, random crop/resize, color jitter (brightness, contrast, saturation).

D11. Five 3x3 filters with stride 2 applied to an 11x11x30 input. What is the output shape and parameter count? A: H_out = floor((11-3)/2)+1 = 5. W_out = 5. C_out = 5. Output = 5x5x5. Params = (3*330 + 1)*5 = 271*5 = 1355.

D12. Consider CNN: Conv(32,3x3)->ReLU->Conv(32,3x3)->ReLU->MaxPool(2)->Flatten->Linear(128)->ReLU->Dropout(0.5)->Linear(10). Input 28x28 grayscale. What is the shape after each layer? A: Input: 1x28x28. Conv1: 32x26x26. Conv2: 32x24x24. MaxPool: 32x12x12. Flatten: 4608. Linear1: 128. Dropout: 128. Linear2: 10.

D13. How many parameters does the network in D12 have? A: Conv1: (3*31+1)*32 = 320. Conv2: (3*332+1)*32 = 9248. Linear1: 4608*128+128 = 589,952. Linear2: 128*10+10 = 1290. Total = 600,810.

D14. A conv layer has 64 filters of size 5x5 applied to a 128-channel input. How many parameters? What about depthwise separable? A: Standard: (5*5*128+1)*64 = 204,864. Depthwise separable: depthwise (5*5*1+1)*128 = 3,328 + pointwise (1*1*128+1)*64 = 8,256. Total = 11,584. Dramatically fewer parameters!

D15. If you replace all max pooling layers with stride-2 convolutions, what happens to trainable parameters? A: Max pooling has zero parameters. Strided convolutions have (K^2*C_in+1)*C_out parameters. Replacing pooling with strided conv significantly increases parameters, but the downsampling becomes learnable and more expressive.

D16. A 224x224 RGB image goes through: Conv(64,7x7,stride=2,padding=3)->MaxPool(3x3,stride=2)->Conv(64,3x3,padding=1). What are the spatial dimensions? A: After Conv1: floor((224-7+6)/2)+1 = 112. After Pool: floor((112-3)/2)+1 = 55. After Conv2: floor((55-3+2)/1)+1 = 55 (same padding).

D17. You have a pre-trained ResNet-50 on ImageNet and want to classify 5 flower types with 500 images. Describe your transfer learning strategy. A: 1) Load pre-trained ResNet-50. 2) Replace final FC (1000 outputs) with Linear(5). 3) Stage 1: freeze backbone, train head with moderate LR (0.001). 4) Stage 2: unfreeze backbone, use small LR (1e-5) and weight decay. 5) Use data augmentation to prevent overfitting.

D18. What happens if you set the learning rate too high during CNN training? A: Loss oscillates wildly or diverges to infinity. The optimizer takes steps so large it jumps over minima. With batch norm, extremely high LR can cause numerical instability.

D19. What happens if you forget to call optimizer.zero_grad() in PyTorch? A: Gradients accumulate across iterations. Weights are updated with the sum of current and all previous gradients, leading to incorrect, exploding updates and broken training.

D20. Why might a CNN overfit even with dropout? A: Dropout probability too low. Network too deep/wide for dataset size. Insufficient data augmentation. Training too many epochs without early stopping. Dropout may only be in FC layers, allowing conv layers to overfit.

D21. What is the receptive field of a neuron, and how does it grow with depth? A: The receptive field is the region in the original image that influences a neuron's output. With stride 1 and no dilation: RF_L = RF_{L-1} + (K_L - 1) * J_{L-1}, where J is cumulative stride.

D22. If you want a 15x15 receptive field using only 3x3 convolutions with stride 1, how many layers do you need? A: Each 3x3 layer adds 2 to RF. Starting from 1: 1 + 2n = 15 -> n = 7 layers.

D23. What is Global Average Pooling (GAP), and why is it useful? A: GAP averages each feature map to a single value, replacing fully connected layers. It drastically reduces parameters, prevents overfitting, and makes the network accept variable input sizes. Used in ResNet and MobileNet.

D24. What is the difference between feature extraction and fine-tuning in transfer learning? A: Feature extraction: freeze pre-trained backbone, train only new classifier head. Best for small, similar datasets. Fine-tuning: unfreeze some/all layers and train with small LR. Best for large or very different datasets.

D25. What is the role of 1x1 convolutions in CNNs? A: 1x1 convolutions act as channel-wise linear transformations. They can: reduce/increase channels (bottleneck/expansion), add non-linearity without changing spatial dimensions, and enable depthwise separable convolutions. Used heavily in ResNet bottlenecks and MobileNet.

D26. What was AlexNet's key innovation that enabled deep CNN training? A: ReLU activation (avoided vanishing gradient of sigmoid/tanh), Dropout for regularization, GPU training for speed, and data augmentation (random crops, flips, PCA color jittering). Won ImageNet 2012 by 10.8 percentage points.

D27. How does ResNet enable training of networks with 100+ layers? A: Residual blocks learn F(x) = H(x) - x instead of H(x) directly, with output y = F(x) + x. Skip connections provide an identity pathway. Gradients can flow directly through the network via the +1 term in the derivative, preventing vanishing gradients.

D28. What is the difference between an identity block and a convolutional block in ResNet? A: Identity block: F(x) + x, used when input and output have the same dimensions. Convolutional block: F(x) + W_s*x, where W_s is a projection shortcut (1x1 conv) used when dimensions change (e.g., after pooling or when changing channels).

D29. What is L2 regularization (weight decay), and how does it prevent overfitting? A: L2 regularization adds lambda * sum(w^2) to the loss. It penalizes large weights, encouraging the network to distribute weights more evenly and preventing any single feature from dominating. This reduces model complexity and overfitting.

D30. True or False: Max pooling layers have trainable parameters. A: False. Max pooling performs a fixed operation (taking the maximum). There are no learnable weights or biases. Average pooling also has no parameters.

D31. True or False: You should always use dropout during model evaluation/inference. A: False. Dropout is a training-only regularization technique. During inference, all neurons should be active to use the full model capacity. Applying dropout at test time would produce random, inconsistent predictions.

D32. True or False: Replacing max pooling with strided convolution always increases the number of trainable parameters. A: True. Max pooling has zero parameters. Strided convolution has (K^2*C_in+1)*C_out parameters. Therefore, replacement always adds trainable parameters.

D33. What is an epoch, and how does it differ from an iteration? A: An epoch is one complete pass over the entire training dataset. An iteration is one parameter update step, typically on a mini-batch of size B. One epoch = N/B iterations, where N is the dataset size.

D34. Why is cross-entropy loss preferred over MSE for classification? A: Cross-entropy penalizes confident wrong predictions much more heavily than MSE. Its gradient is well-behaved and directly tied to the softmax output, leading to faster convergence and better classification boundaries.

D35. What happens to gradients during backpropagation through a ReLU activation? A: For positive inputs, gradient passes through unchanged (multiplied by 1). For negative inputs, gradient is blocked (multiplied by 0). This sparsity can speed up computation but may cause 'dying ReLU' if too many neurons are always negative.

D36. How does the choice of batch size affect training? A: Small batch sizes: noisier gradients, better generalization, slower training (less GPU utilization). Large batch sizes: smoother gradients, faster training, but may converge to sharp minima that generalize poorly. Very large batches may require learning rate scaling.

D37. What is early stopping, and why is it effective against overfitting? A: Early stopping monitors validation loss and stops training when it stops improving for a patience number of epochs. It prevents the model from continuing to fit noise in the training data after it has learned the general patterns.

D38. What is the difference between padding='same' and padding='valid' in TensorFlow/Keras terms? A: 'Same': pad so output spatial dimensions equal input dimensions (for stride 1). 'Valid': no padding; output shrinks by (K-1) per layer. In PyTorch, padding is specified numerically.

D39. What is a bottleneck block in ResNet, and why does it use 1x1 convolutions? A: A bottleneck block has three layers: 1x1 (reduce channels), 3x3 (spatial processing), 1x1 (restore channels). The 1x1 convolutions reduce computational cost by decreasing the number of channels before the expensive 3x3 convolution.

D40. What is the role of the loss function in training a neural network? A: The loss function quantifies the difference between the network's predictions and the ground truth. During backpropagation, gradients of the loss with respect to each parameter are computed, guiding weight updates to minimize this difference.

D41. How does SGD with momentum differ from vanilla SGD? A: Momentum accumulates a velocity vector: v_t = beta*v_{t-1} + gradient. This helps accelerate in consistent directions, dampen oscillations, and escape shallow local minima. Vanilla SGD uses only the current gradient.

D42. What is Adam optimizer, and what makes it 'adaptive'? A: Adam maintains moving averages of both the gradient (first moment, like momentum) and the squared gradient (second moment, like RMSprop). It adapts the learning rate per parameter based on these estimates, performing larger updates for sparse gradients and smaller updates for frequent ones.

D43. Why might you choose SGD over Adam for training a very deep network from scratch? A: SGD with momentum often generalizes better than Adam and can find wider minima. Adam's adaptive learning rates may cause it to converge too quickly to sharp minima. However, Adam converges faster initially and requires less tuning of the learning rate.

D44. What is the output shape of a Conv2D layer with 64 filters, kernel 3x3, stride 2, padding 1, applied to 32x32x3 input? A: H_out = floor((32 - 3 + 2*1)/2) + 1 = floor(31/2) + 1 = 15 + 1 = 16. Output: 16x16x64.

D45. What is the total parameter count for the layer in D44? A: Params = (3*3*3 + 1)*64 = (27+1)*64 = 28*64 = 1792.

D46. What are the 5 steps of Canny edge detection? A: 1) Gaussian smoothing to reduce noise. 2) Compute gradient magnitude and direction with Sobel. 3) Non-maximum suppression to thin edges. 4) Double thresholding to classify strong and weak edges. 5) Hysteresis edge tracking to connect weak edges to strong ones.

D47. Why does Canny use hysteresis instead of a single threshold? A: A single threshold either misses weak edges (too high) or includes too much noise (too low). Hysteresis uses a high threshold for certain edges and a low threshold for candidate edges, keeping only weak edges that connect to strong ones. This produces continuous contours while suppressing noise.

D48. A model has 95% accuracy but only 30% recall on the minority class. Is accuracy a good metric here? What should you use instead? A: No. With class imbalance, high accuracy can hide poor minority-class performance. Use F1 score (harmonic mean of precision and recall), or report precision and recall separately. In medical screening, recall is critical (don't miss positive cases).

D49. Explain the difference between Xavier (Glorot) and He (Kaiming) weight initialization. When do you use each? A: Xavier initialization uses variance 2/(n_in + n_out) and is designed for sigmoid/tanh activations. He initialization uses variance 2/n_in and is designed for ReLU activations, accounting for the fact that ReLU zeros out half the activations. Use Xavier for sigmoid/tanh, He for ReLU.

D50. What is the purpose of a learning rate scheduler, and name two common types. A: A constant LR may be too large late in training (causing oscillation) or too small early (slow convergence). Schedulers adapt LR over time. Common types: StepLR (reduce by factor every N epochs), ReduceLROnPlateau (reduce when validation loss stops improving), Cosine Annealing (smooth cosine decay).

D51. Consider a CNN with Conv1: 3x3, 32 filters, stride 2, padding 1 on 224x224x3 input. Conv2: 3x3, 64 filters, stride 2, padding 1. MaxPool: 2x2, stride 2. How many parameters in each layer? A: - Conv1: (3*3*3 + 1)*32 = (27+1)*32 = 896 parameters. Output: 112x112x32. - Conv2: (3*3*32 + 1)*64 = (288+1)*64 = 18,496 parameters. Output: 56x56x64. - MaxPool: 0 parameters. Output: 28x28x64.

D52. Would the parameter count change if the input were grayscale (1 channel) instead of RGB (3 channels)? Explain. A: Yes, the first conv layer would change. With grayscale: Conv1 params = (3*3*1 + 1)*32 = 320 instead of 896. However, all subsequent layers would remain the SAME because they depend only on the number of output channels from the previous layer, not the original input channels. The downstream architecture is unaffected by input channels after the first layer.

D53. What would happen to the number of trainable parameters if you replaced a MaxPool layer with a Conv2D + ReLU layer? A: MaxPool has **0 parameters** — it's a fixed operation (take maximum). A Conv2D + ReLU layer has `(K*K*C_in + 1)*C_out` parameters. Replacing MaxPool with Conv+ReLU would drastically increase parameters, especially before FC layers where spatial dimensions are still large. This increases model capacity but also risk of overfitting and slower training. The trade-off: MaxPool reduces dimensions cheaply; Conv+ReLU learns downsampling but at high parameter cost.

D54. What are the first layers of a CNN learning, and what are the last layers learning? Give examples. A: First conv layers learn low-level, generic visual primitives: oriented edge detectors (various angles), color blobs, Gabor-like patterns, and intensity gradients. These are universal across tasks and datasets. Example: a filter might detect a vertical edge or a blue-yellow color transition.

Last layers (deep conv + FC) learn task-specific, high-level features: object parts (wheels, eyes, door handles) and class-discriminative combinations. Example: a neuron in the penultimate layer might activate for "dog face" or "car front". The final FC layer converts these into class scores. These must be replaced or heavily fine-tuned for new tasks.

D55. Is it possible to replace the fully connected layers of a CNN with an SVM classifier? What are the pros and cons? A: Yes. Extract features from the CNN's final conv layer (before FC), flatten them, and train an SVM on these features instead of the FC layers.

Pros: SVMs with RBF kernels can model non-linear decision boundaries and may generalize better on small datasets. They don't require gradient-based training.

Cons: The CNN features are NOT optimized for the SVM objective. End-to-end backpropagation through FC layers adapts conv features specifically for classification, which usually outperforms frozen features + SVM. SVMs also scale poorly to very large datasets and don't naturally handle multi-class problems as elegantly as softmax. Modern practice: fine-tune the entire network end-to-end.

D56. What is the main difference between BatchNorm and LayerNorm? When would you use each? A: BatchNorm normalizes across the batch dimension (computes mean/var over all samples in a batch for each channel). LayerNorm normalizes across the feature dimension (computes mean/var over all channels for EACH sample independently). Use BatchNorm for CNNs with sufficiently large batches. Use LayerNorm for RNNs/Transformers where batch size may be 1 or variable.

D57. What is Focal Loss, and why is it useful for object detection? A: Focal Loss = $-\alpha(1-p)^{\gamma}\log(p)$. It down-weights easy examples (high-confidence correct predictions) so that training focuses on hard examples. In

object detection, most candidate boxes are background (easy negatives), causing class imbalance. Focal Loss prevents these easy negatives from dominating the loss, allowing the model to learn from rare positive examples.

D58. What is the key idea of DenseNet, and how does it differ from ResNet? A: DenseNet connects each layer to ALL subsequent layers (dense connectivity), concatenating features. ResNet uses skip connections that ADD features (x + F(x)). DenseNet CONCATENATES features, enabling maximum feature reuse. DenseNet has fewer parameters but more feature maps per layer. Growth rate k controls new features per layer.

D59. What is compound scaling in EfficientNet? A: Instead of scaling only depth, width, or resolution, EfficientNet scales all three dimensions simultaneously with a fixed ratio ($d=\alpha^\varphi$, $w=\beta^\varphi$, $r=\gamma^\varphi$ where $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$). This ensures balanced scaling — increasing depth alone would saturate quickly; increasing all three proportionally improves accuracy more efficiently.

D60. What is Dice Loss, and when would you prefer it over CrossEntropy for segmentation? A: Dice Loss $= 1 - 2|X \cap Y|/(|X|+|Y|)$, measuring overlap between predicted and ground-truth masks. Prefer it over CrossEntropy when classes are highly imbalanced (e.g., medical images where foreground is tiny). CrossEntropy is dominated by the large background class, while Dice Loss directly optimizes overlap and treats both classes more equally.

SECTION D: CONVOLUTIONAL NEURAL NETWORKS (60 Questions)

E1. Why is Faster R-CNN approximately 10x faster than original R-CNN? A: R-CNN runs ~2000 separate CNN forward passes (one per region proposal). Faster R-CNN runs the CNN once on the full image (shared features), uses an internal RPN for proposals, and shares all computation. One forward pass instead of ~2000.

E2. Explain why YOLO is called 'You Only Look Once' and how it differs from Faster R-CNN. A: YOLO divides the image into a grid and predicts bounding boxes and classes for each cell in a single network forward pass. Faster R-CNN is two-stage: RPN proposes regions, then a second stage classifies each. YOLO removes the explicit proposal stage, looking at the image once.

E3. A YOLO model uses 7x7 grid, 2 boxes per cell, 20 classes. How many outputs per image? How many bounding box predictions? A: Total outputs = $7*7*(2*5 + 20) = 49*30 = 1470$. Bounding box predictions = $7*7*2 = 98$.

E4. What is the purpose of skip connections in U-Net? A: During encoding, repeated pooling downsamples spatial resolution and loses fine detail. Skip connections concatenate high-resolution encoder features with decoder features at the same resolution, allowing the decoder to recover precise spatial localization. Without them, output would be blurry.

E5. Why is U-Net a good choice for biomedical image segmentation? A: 1) Skip connections preserve fine cellular boundaries. 2) Works well with small datasets (common in medicine) due to architectural inductive biases. 3) Encoder-decoder captures both global context and local detail. 4) End-to-end trainable with pixel-wise loss.

E6. What is the difference between RoI Pooling and RoI Align? Why does Mask R-CNN use Align? A: RoI Pooling quantizes RoI boundaries to integers, causing coarse misalignment. RoI Align uses bilinear interpolation at continuous locations without quantization, preserving spatial precision. Mask R-CNN needs pixel-accurate masks, so RoI Align is essential.

E7. What is Non-Maximum Suppression (NMS), and why is it needed? A: Object detectors often predict multiple overlapping boxes for the same object. NMS sorts boxes by confidence, keeps the highest-confidence box, and removes all overlapping boxes with IoU > threshold. This produces one detection per object.

E8. For blood cell microscopy where cells touch, why is Mask R-CNN preferred over basic semantic segmentation? A: Semantic segmentation labels all blood cells with the same class, merging touching cells into one blob. Mask R-CNN performs instance segmentation: it detects each cell individually with a bounding box and predicts a separate binary mask per instance, physically separating touching cells.

E9. What is the role of the objectness score in YOLO output? A: The confidence score P(object)*IoU(pred, truth) indicates both the probability that a box contains an object and how accurate the box is. During inference, low-confidence boxes are filtered out before NMS, reducing false positives.

E10. Why can a Fully Convolutional Network (FCN) accept input images of arbitrary size, while a network with FC layers cannot? A: Convolutions and pooling are translation-invariant and operate locally; their output size simply scales with input size. FC layers require a fixed-size input vector. FCNs replace FC with 1x1 convolutions, maintaining spatial structure.

E11. You need to detect 10 object classes in 200x200 images using YOLO with 7x7 grid and 2 boxes per cell. How many output values per image? A: $7*7*(2*5 + 10) = 49*20 = 980$ output values per image.

E12. A segmentation network outputs a 224x224 mask with 21 classes (20 objects + background). How many channels does the final conv layer have? A: 21 channels — one per class. Each channel contains logits/scores for that class at each pixel. Softmax is applied across the channel dimension per pixel.

E13. Design a system to count cars in a parking lot from a static camera. Which detector and why? A: Use YOLOv8 or similar one-stage detector. Real-time performance is needed for monitoring. The fixed scene allows fine-tuning a lightweight variant. Post-process with NMS and tracking (SORT/DeepSORT) for consistent counts across frames.

E14. What is IoU (Intersection over Union), and why is 0.5 commonly used as a detection threshold? A: IoU = Area(Intersection) / Area(Union). It measures overlap between predicted and ground-truth boxes. IoU > 0.5 indicates substantial overlap and is a standard threshold for considering a detection correct, balancing precision and recall.

E15. What is mAP (mean Average Precision), and how is it computed? A: Compute Precision-Recall curve at various IoU thresholds for each class. AP = area under PR curve for one class. mAP = mean of AP over all classes. mAP@0.5 averages at IoU=0.5; mAP@0.5:0.95 averages across multiple thresholds (COCO standard).

E16. What is the main drawback of R-CNN that Fast R-CNN solves? A: R-CNN extracts ~2000 region proposals and runs a separate CNN forward pass for each, making it extremely slow. Fast R-CNN runs the CNN once on the full image, sharing convolutional features across all proposals, then uses RoI pooling.

E17. What is the role of the Region Proposal Network (RPN) in Faster R-CNN? A: The RPN slides a small network over the shared convolutional feature map and predicts objectness scores and bounding box refinements for anchor boxes at each location. It replaces external Selective Search, making the network end-to-end trainable.

E18. What are anchor boxes in object detection? A: Anchor boxes are pre-defined bounding boxes of various aspect ratios and scales placed at each spatial location in the feature map. Detectors (RPN, YOLOv2+) predict offsets relative to these anchors rather than absolute coordinates.

E19. Why did early YOLO versions struggle with small objects? A: YOLOv1 used a coarse grid (7x7) and each cell predicted only 2 boxes. Small objects might fall entirely within one grid cell or be dwarfed by the cell size, making localization imprecise. Later versions added multi-scale predictions and finer grids.

E20. What is atrous (dilated) convolution, and why does DeepLab use it? A: Atrous convolution inserts holes between kernel weights, increasing the receptive field without increasing parameters or losing resolution. DeepLab uses Atrous Spatial Pyramid Pooling (ASPP) to capture multi-scale context while maintaining high-resolution feature maps.

E21. What is the difference between semantic segmentation and panoptic segmentation? A: Semantic segmentation classifies every pixel into a class label without distinguishing instances. Panoptic segmentation combines semantic segmentation for ‘stuff’ classes (sky, road) with instance segmentation for ‘thing’ classes (people, cars) into a unified output.

E22. Why is instance segmentation harder than semantic segmentation? A: Semantic segmentation only needs to classify each pixel’s category. Instance segmentation must additionally separate individual objects of the same class, which requires handling occlusions, overlapping objects, and ambiguous boundaries between touching instances.

E23. What is the main advantage of two-stage detectors (Faster R-CNN) over one-stage (YOLO)? A: Two-stage detectors typically achieve higher accuracy, especially on small objects and dense scenes, because the region proposal stage filters out most background before classification. One-stage detectors trade some accuracy for much greater speed.

E24. How does feature pyramid network (FPN) help with multi-scale object detection? A: FPN builds a pyramid of feature maps at different resolutions by combining high-resolution, semantically weak features with low-resolution, semantically strong features via lateral connections. This enables detecting small objects in high-res maps and large objects in low-res maps.

E25. What is the role of the classification and regression heads in Faster R-CNN? A: The classification head predicts the probability distribution over C object classes (plus background). The regression head predicts bounding box offsets (dx, dy, dw, dh) relative to the proposal to refine its coordinates.

E26. In YOLO, what do the 5 values (x, y, w, h, confidence) represent for each bounding box? A: x,y = center coordinates of the box relative to the grid cell (0-1). w,h = width and height relative to the full image (0-1). confidence = P(object) * IoU(predicted_box, ground_truth).

E27. What happens during YOLO loss computation? A: The loss is a weighted sum of: coordinate loss (MSE for x,y,w,h), confidence loss (MSE for objectness scores), and classification loss (cross-entropy for class probabilities). Only grid cells responsible for ground-truth objects contribute to most losses.

E28. Why does Mask R-CNN use a binary mask per RoI instead of predicting masks directly from the feature map? A: Predicting a mask per RoI allows instance-specific segmentation. Each detected object gets its own mask branch output, enabling separation of overlapping instances. Direct feature-map prediction would produce semantic segmentation without instance separation.

E29. What is the difference between bounding box regression in R-CNN and Fast R-CNN? A: R-CNN performs bounding box regression as a post-processing step after SVM classification. Fast R-CNN integrates bounding box regression into the network itself, jointly optimizing classification and regression in a multi-task loss.

E30. True or False: U-Net can be used for image generation tasks, not just segmentation. A: True. The encoder-decoder architecture with skip connections is essentially an image-to-image mapping network. It is used in diffusion models, VAEs, and super-resolution as a backbone generator.

E31. What is the main challenge of using FCNs for instance segmentation? A: FCNs produce semantic segmentation (one label per pixel) but cannot distinguish between different instances of the same class. Additional mechanisms (detection + mask branches like Mask R-CNN, or clustering post-processing) are needed for instance separation.

E32. How does transposed convolution (deconvolution) work in decoder networks? A: Transposed convolution learns to upsample by inserting zeros between input elements and applying a convolution. Unlike fixed upsampling (nearest neighbor, bilinear), it has learnable parameters that can adapt to recover spatial detail during training.

E33. What is the role of the encoder in a segmentation network like U-Net? A: The encoder extracts hierarchical features through repeated convolutions and pooling, capturing increasingly abstract context (what objects are present) while reducing spatial resolution. The bottleneck contains the highest semantic information.

E34. Why might you prefer Mask R-CNN over U-Net for segmenting overlapping objects? A: U-Net produces semantic segmentation where all objects of the same class share one mask, causing overlapping objects to merge. Mask R-CNN detects each object individually and predicts a separate binary mask per instance, naturally separating overlaps.

E35. What is the COCO dataset's standard evaluation metric, and how does it differ from Pascal VOC's? A: COCO uses mAP@0.5:0.95 (averaged over IoU thresholds from 0.5 to 0.95 in steps of 0.05), which is stricter. Pascal VOC uses mAP@0.5 only. COCO also evaluates across small, medium, and large objects separately.

E36. What is DETR, and how does it differ from Faster R-CNN and YOLO? A: DETR (Detection Transformer) treats object detection as a set prediction problem using a Transformer encoder-decoder. Unlike Faster R-CNN (two-stage with anchors/RPN) and YOLO (one-stage with grid/anchors), DETR has NO anchors, NO NMS, and NO region proposals. It uses learned object queries and Hungarian matching for end-to-end detection.

E37. Explain the DETR architecture: backbone, encoder, decoder, and prediction heads. A: 1) CNN backbone (ResNet) extracts features. 2) Transformer encoder with self-attention reasons globally over the image. 3) Transformer decoder processes learned object queries to predict objects. 4) FFN heads predict class label (including “no object”) and bounding box coordinates for each query.

E38. What are ‘object queries’ in DETR, and why are they necessary? A: Object queries are learned embedding vectors (typically 100) that act as “slots” for detecting objects. Each query attends to the encoded image features via cross-attention and predicts one object. They replace anchor boxes and region proposals. Fixed number of queries sets the maximum detections per image.

E39. How does DETR handle the matching between predictions and ground truth during training? A: DETR produces a fixed set of predictions (e.g., 100). Ground truth may have fewer objects. It uses the Hungarian algorithm to find the optimal bipartite matching between predictions and ground truth. Cost = classification error + L1 box distance + generalized IoU loss. Unmatched predictions are penalized as “no object”.

E40. What are the main advantages of DETR over traditional detectors? A: 1) True end-to-end: no anchors, no NMS, no hand-designed components. 2) Global reasoning via self-attention over the entire image. 3) Simpler pipeline with fewer hyperparameters. 4) Naturally handles overlapping objects without NMS because each query predicts one distinct object.

E41. What are the main limitations of DETR? A: 1) Slow convergence: requires 500+ epochs vs ~12 for Faster R-CNN. 2) Poor small object detection due to coarse backbone feature maps. 3) Fixed number of queries limits maximum detections. 4) Quadratic complexity O(N^2) from self-attention. 5) Higher memory requirements.

E42. True or False: DETR requires Non-Maximum Suppression (NMS) during inference. A: False. DETR does not need NMS because each object query is trained to predict a distinct object. The set prediction formulation and Hungarian matching naturally avoid duplicate detections without explicit NMS.

E43. What is Deformable DETR, and how does it address DETR’s limitations? A: Deformable DETR replaces full self-attention with deformable attention, which only attends to a small set of sampled locations around a reference point. This reduces complexity from O(N^2) to O(N), speeds up convergence, and improves small object detection by using multi-scale features.

E44. Compare DETR with Faster R-CNN in terms of: anchors, NMS, global context, and training speed. A: Faster R-CNN: uses anchors + RPN, requires NMS, limited global context (local receptive fields), trains fast (~12 epochs). DETR: no anchors, no NMS needed, full global context via self-attention, trains very slowly (500+ epochs).

E45. A YOLO model is trained on 200x200 images to detect 10 object classes. Using a 7x7 grid with 2 boxes per cell (YOLOv1 style), how many outputs does YOLO produce per image? Justify. A: YOLO outputs per image = $S^2 \times (B \times 5 + C)$ where S=7, B=2, C=10. → $7 \times 7 \times (2 \times 5 + 10) = 49 \times 20 =$ **980 outputs total**. Each cell outputs: 2 bounding boxes (each with x, y, w, h, confidence = 5 values) + 10 class probabilities. → Per cell: 2×5 + 10 = 20 values. → Total bounding box predictions: 7×7×2 = 98 boxes.

E46. The U-Net was initially proposed for biomedical image segmentation but has become a key architecture for image generation tasks. Why is this the case? A: U-Net's encoder-decoder architecture with skip connections is fundamentally an **image-to-image mapping** network. The encoder compresses the input into a latent representation (capturing semantics), and the decoder reconstructs an output image (segmentation mask, generated image, super-resolved image, etc.). Skip connections preserve fine spatial details that would otherwise be lost during downsampling. This makes it ideal for any task requiring pixel-accurate output: segmentation, image generation (as a backbone in VAEs/diffusion models), denoising, inpainting, and super-resolution. The symmetry and skip connections make it a universal "U-shaped" image transformer.

E47. Faster R-CNN is approximately 10x faster than R-CNN. Explain exactly why. A: R-CNN runs ~2000 forward passes through the CNN (one per region proposal from Selective Search). Each crop is warped and fed independently → massive redundancy (overlapping regions recompute conv features repeatedly).

Faster R-CNN: 1. Runs the CNN **once** on the full image → shared feature map. 2. Uses a **Region Proposal Network (RPN)** inside the network (learned, not external Selective Search). 3. Shares all convolutional computation between proposal generation and classification. 4. Proposals are classified by cropping from the shared feature map (RoI pooling), not from raw pixels.

Result: ~1 forward pass instead of ~2000 → roughly **10× faster** (R-CNN ~50s/image, Faster R-CNN ~5s/image on CPU; even greater speedup on GPU with batching).

E48. Explain why YOLO is called "You Only Look Once" and why this is a differentiating factor from Faster R-CNN. A: In R-CNN family (including Faster R-CNN), detection is **two-stage**: first propose regions (RPN), then classify each region separately. This is like "looking twice" — once to find potential objects, once to classify them.

YOLO is **one-stage**: it divides the image into a grid and simultaneously predicts bounding boxes, confidence scores, and class probabilities for ALL cells in a **single network evaluation** (one forward pass). It literally looks at the image only once.

Differentiating factor: this makes YOLO extremely fast (real-time capable at 45+ FPS for YOLOv1, 140+ FPS for later versions) while Faster R-CNN is slower but more accurate. YOLO sacrifices some accuracy (especially on small objects in early versions) for speed.

SECTION E: OBJECT DETECTION AND SEGMENTATION (48 Questions)

F1. Can a GAN be used to solve class imbalance? If yes, how? A: Yes. Train a conditional GAN on the minority class to generate realistic synthetic samples. Augment the training set with these samples until classes are balanced. Then train the classifier on the balanced dataset. Useful in medical imaging where disease samples are rare.

F2. Explain the minimax objective of GANs. What does D maximize and G minimize? A: min_G max_D E[log D(x)] + E[log(1 - D(G(z)))]. D maximizes probability of correctly classifying real and fake. G minimizes the chance that D identifies fakes, equivalently maximizing log(D(G(z))).

F3. What is mode collapse, and why is it problematic? A: Mode collapse occurs when the generator learns to produce a limited variety of outputs (or even a single output) that fools the discriminator. It ignores large parts of the true data distribution, resulting in lack of diversity in generated samples.

F4. Explain why cycle consistency loss is critical in CycleGAN. A: Without cycle consistency, G: X->Y could learn an arbitrary mapping that ignores input content (e.g., always generating the same zebra regardless of horse pose). Cycle loss enforces F(G(x)) ~ x, meaning the translated image must retain enough information to reconstruct the original, ensuring semantic preservation.

F5. What is the difference between a standard GAN and a conditional GAN (cGAN)? A: In a cGAN, both generator and discriminator receive auxiliary conditioning information y (class label, text, image). G(z,y) generates samples of class y. This enables controlled generation.

F6. Why are GANs notoriously difficult to train compared to standard classification networks? A: 1) Non-convex adversarial game with no single loss minimum. 2) Vanishing gradients if D is too good. 3) Mode collapse. 4) Balancing act between D and G strengths. 5) No clear quality metric like classification accuracy.

F7. True or False: In a GAN, the generator and discriminator are always trained simultaneously with the same learning rate. A: False. While trained alternately, they often require different learning rates or update frequencies. D is frequently trained more steps per G step to maintain balance. Same rates can cause one to dominate.

F8. What is the Wasserstein GAN (WGAN), and how does it improve training stability? A: WGAN replaces the Jensen-Shannon divergence with the Earth Mover's (Wasserstein) distance. This provides meaningful gradients even when D is near optimal, preventing vanishing gradients and mode collapse issues.

F9. What is the identity loss in CycleGAN? A: The identity loss encourages G(x) ~ x when x is already in the target domain Y. It ensures the generator preserves images that don't need translation, preventing unnecessary modifications.

F10. What are common applications of GANs in computer vision? A: Image generation, image-to-image translation, super-resolution, data augmentation/class imbalance, inpainting (filling missing regions), style transfer, and text-to-image synthesis.

F11. In a GAN, what does it mean when the discriminator outputs 0.5 for all inputs? A: Theoretically, this indicates Nash equilibrium: the discriminator cannot distinguish real from fake, and the generator has perfectly learned the data distribution. In practice, it may also indicate training instability or a collapsed generator.

F12. How does a conditional GAN enable controlled image generation? A: By feeding class labels or other conditioning variables to both G and D, the generator learns to produce samples conditioned on the input. For example, providing digit class '7' to G(z, '7') generates images of the digit 7.

F13. What is the difference between pixel-wise loss and adversarial loss in image generation? **A**: Pixel-wise loss (e.g., L1, MSE) compares generated and target images pixel by pixel, often producing blurry results. Adversarial loss uses the discriminator to evaluate realism, encouraging high-frequency details and sharp outputs.

F14. Why might a GAN generator produce blurry images? **A**: If the discriminator is too weak, the generator has no incentive to produce sharp details. Also, if the loss function includes strong pixel-wise MSE terms, the generator converges to the mean of possible outputs (blurry average).

F15. What is the role of noise vector z in the GAN generator? **A**: z is a random latent vector sampled from a simple distribution (e.g., Gaussian). It provides the generator with a source of randomness to generate diverse outputs. Different z values map to different points in the learned data manifold.

F16. How can you evaluate the quality of generated images without human judgment? **A**: Common metrics: Inception Score (IS) measures diversity and classifiability; Frechet Inception Distance (FID) compares feature distributions of real and generated images using a pre-trained Inception network. Lower FID = better quality.

F17. What is pix2pix, and how does it relate to conditional GANs? **A**: pix2pix is a conditional GAN for paired image-to-image translation (e.g., sketches to photos). It uses a U-Net generator and patch-based discriminator. The conditioning is the input image, and the target is the paired output image.

F18. What is the difference between paired and unpaired image-to-image translation? **A**: Paired (pix2pix): training data contains aligned input-output pairs. Unpaired (CycleGAN): training data has two separate collections (e.g., all horse photos and all zebra photos) with no correspondences. CycleGAN uses cycle consistency to learn without pairs.

F19. What is spectral normalization, and why is it used in some GANs? **A**: Spectral normalization normalizes the spectral norm (largest singular value) of weight matrices to 1. It constrains the Lipschitz constant of the discriminator, stabilizing training and preventing the discriminator from becoming too powerful too quickly.

F20. Can GANs be used for anomaly detection? If so, how? **A**: Yes. Train a GAN or autoencoder on normal data. At test time, anomalies will be poorly reconstructed or will have high discriminator rejection probability. The reconstruction error or discriminator score serves as an anomaly score.

F21. What is progressive growing of GANs (ProGAN)? **A**: ProGAN starts training with low-resolution images (4x4) and progressively adds layers to increase resolution (8x8, 16x16, etc.). This stabilizes training by first learning coarse structure before adding fine details, leading to high-quality image generation.

F22. What is the role of the discriminator in a GAN beyond just saying ‘real’ or ‘fake’? **A**: The discriminator provides a learning signal (gradient) to the generator. Its intermediate features can also be used for representation learning. In some architectures, the discriminator guides the generator toward more realistic outputs by backpropagating classification error.

F23. How does minibatch discrimination help prevent mode collapse? **A**: Minibatch discrimination allows the discriminator to look at multiple samples simultaneously and penalize lack of diversity within a batch. This discourages the generator from producing identical outputs, promoting coverage of multiple modes.

F24. What is the difference between a GAN and a VAE (Variational Autoencoder)? **A**: A VAE is a probabilistic model with an explicit likelihood (reconstruction + KL divergence), producing blurry but stable outputs. A GAN uses an implicit density with adversarial training, producing sharper but harder-to-train outputs. VAEs have an explicit latent space; GANs learn an implicit one.

F25. In CycleGAN, what would happen if you trained only G: X->Y without F: Y->X and without cycle loss? **A**: The generator G could learn an arbitrary mapping that ignores the input content (e.g., map every horse to the same zebra regardless of pose). Without the backward cycle and cycle loss, there is no constraint forcing semantic preservation.

F26. What are the key architectural constraints of DCGAN that make it stable? **A**: 1) Replace pooling with strided convolutions. 2) Use batch normalization in both G and D. 3) Remove fully connected hidden layers for deeper architectures. 4) Use ReLU in G (Tanh at output) and LeakyReLU in D. These constraints prevent mode collapse and vanishing gradients.

F27. How does ProGAN progressively grow the generator and discriminator? **A**: Training starts at low resolution (4×4). As training stabilizes, new layers are faded in to double the resolution ($4 \rightarrow 8 \rightarrow 16 \rightarrow \ldots \rightarrow 1024$). The new layers are smoothly introduced via parameter interpolation. This allows the network to learn coarse structure first before adding fine details, stabilizing training for very high-resolution generation.

F28. What is the difference between Inception Score (IS) and FID for evaluating GANs? **A**: IS measures whether generated images are classifiable (low entropy p(y|x)) and diverse (high entropy p(y)). It doesn't compare to real data. FID computes the Fréchet distance between real and generated feature distributions (using Inception features). Lower FID = closer to real data distribution. FID is more robust and the standard metric.

SECTION F: GENERATIVE ADVERSARIAL NETWORKS (28 Questions)

G1. For a patch in a Vision Transformer corresponding to a cat's eye, attention weights show strong connections to distant patches (ears, whiskers) but weak to nearby background. What does this suggest? **A**: The model learned semantic grouping and object coherence. It recognizes that eye, ears, and whiskers are parts of the same semantic object. This demonstrates that attention captures long-range semantic dependencies, not just spatial proximity.

G2. Is it easier to combine image and text using a CNN or a Transformer? Explain. **A**: Transformers are easier. Both images (patch tokens) and text (word tokens) can be tokenized into sequences. Self-attention naturally learns cross-modal relationships by allowing any token to attend to any other. CNNs operate on fixed spatial grids; fusing text requires specially designed adapters.

G3. What is the computational complexity of self-attention, and why is it a challenge for high-resolution images? **A**: O(N^2) where N is the number of tokens. For HxW image with patch size P, N = (H*W)/P^2. A

1024x1024 image with 16x16 patches has N=4096, requiring a 4096x4096 attention matrix (~16M entries). Much heavier than CNNs for high resolution.

G4. Why do Vision Transformers need position embeddings, while CNNs do not? A: Self-attention is permutation-invariant: it treats input as an unordered set. Without position embeddings, shuffling patches would produce the same output. CNNs inherently encode position through the spatial structure of convolutions (filters at specific positions process specific regions).

G5. Explain the role of the [CLS] token in ViT. A: The classification token is a learnable embedding prepended to the patch sequence. After passing through the Transformer, its representation aggregates global information from all patches via self-attention. The final [CLS] output is fed to an MLP head for classification, mirroring BERT's [CLS] token.

G6. Compare CNNs and Vision Transformers in terms of inductive bias and data efficiency. A: CNNs have strong inductive biases: locality, translation invariance, hierarchical composition. They learn well from small datasets. Transformers have minimal inductive bias; they must learn spatial relationships from scratch. This makes them data-hungry but more flexible with large pre-training.

G7. If you remove position embeddings from a Vision Transformer, what happens? A: Performance drops dramatically. The model becomes permutation-invariant and cannot distinguish spatial relationships. It would not know that a top-left patch differs from a bottom-right patch. Precise spatial reasoning is lost.

G8. A student claims Transformers will completely replace CNNs for all vision tasks. Do you agree? A: Not completely. Arguments for: state-of-the-art on many benchmarks, powerful global attention, easier multi-modal fusion. Arguments against: CNNs more efficient for high-resolution inputs, better with small data, simpler to train/deploy on edge devices. Hybrid models may be the future.

G9. What is multi-head self-attention, and why is it useful? A: Multi-head attention runs the attention mechanism multiple times in parallel with different learned linear projections of Q, K, V. Each head can attend to different representation subspaces (e.g., one head attends to syntax, another to semantics). Outputs are concatenated and projected.

G10. What is the purpose of scaling the attention scores by sqrt(d_k) in the attention formula? A: For large d_k, dot products grow in magnitude. Large values push the softmax function into regions with extremely small gradients (saturation). Scaling by sqrt(d_k) keeps dot products in a reasonable range, preventing vanishing gradients during backpropagation.

G11. Describe the full pipeline of a Vision Transformer from raw image to classification. A: 1) Split image into non-overlapping patches (e.g., 16x16). 2) Flatten and linearly embed each patch to dimension D. 3) Add position embeddings. 4) Prepend learnable [CLS] token. 5) Pass through Transformer encoder (multi-head attention + MLP + LayerNorm + residuals). 6) Classify using [CLS] output.

G12. What is the main advantage of Transformers for multi-modal tasks like image captioning? A: Both images (as patch tokens) and text (as word tokens) become sequences of embeddings. Self-attention naturally learns cross-modal alignments (e.g., aligning 'cat' token with cat-shaped patches). Simply concatenate sequences and add modality-type embeddings.

G13. How does the attention mechanism in Transformers differ from the receptive field in CNNs? A: CNN receptive field grows with depth and is limited to local neighborhoods in early layers. Transformer attention is global from the first layer: any token can directly attend to any other token, regardless of distance. Attention is also dynamic (content-dependent), while CNN kernels are fixed after training.

G14. What are learned vs sinusoidal position embeddings? A: Learned embeddings are trainable vectors added to each patch position. Sinusoidal embeddings use fixed sine/cosine functions of different frequencies and are not learned. Both encode position; learned may adapt better to data, while sinusoidal generalizes to longer sequences.

G15. True or False: A Vision Transformer can process images of varying sizes without any modification. A: False (mostly). While the Transformer itself is flexible, the patch embedding and position embeddings are typically fixed to a specific number of patches. Handling varying sizes requires adjusting position embeddings or using adaptive approaches. Standard ViT uses fixed input sizes.

G16. What is Layer Normalization, and why is it used in Transformers instead of Batch Normalization? A: Layer Normalization normalizes across the feature dimension for each sample independently. Unlike Batch Norm, it doesn't depend on batch statistics, making it suitable for small batches and sequence models. Transformers use pre-norm or post-norm for training stability.

G17. What is the role of the MLP block in each Transformer encoder layer? A: After attention, the MLP (typically two linear layers with GELU activation) applies a non-linear transformation to each token independently. It increases model capacity and allows the network to learn complex functions of the attention-refined representations.

G18. How does CLIP (Contrastive Language-Image Pre-training) use the Transformer architecture? A: CLIP uses separate text and image encoders (both Transformers or ResNets for images). It learns a joint embedding space where matching image-text pairs are close together and non-matching pairs are far apart, enabling zero-shot classification.

G19. What is the difference between self-attention and cross-attention? A: Self-attention computes attention within the same sequence (Q, K, V all from the same input). Cross-attention computes attention where Q comes from one sequence and K, V come from another (e.g., decoder attending to encoder outputs in sequence-to-sequence models).

G20. Why might a Vision Transformer underperform a CNN when trained from scratch on ImageNet-1k without pre-training? A: ViT lacks the strong inductive biases (locality, translation invariance) of CNNs. On medium-sized datasets like ImageNet-1k, it doesn't have enough data to learn spatial relationships from scratch. It typically needs large-scale pre-training (e.g., JFT-300M) to outperform CNNs.

G21. What is the significance of the attention map visualization in interpretability? A: Attention maps show which patches/tokens the model focuses on when making decisions. They reveal whether the model bases predictions on relevant semantic regions (e.g., attending to a dog's face for dog classification) or on spurious correlations (e.g., background).

G22. How do hybrid models like CoAtNet combine CNNs and Transformers? A: Hybrid models use CNN stem layers (early convolutions) to extract local features and downsample efficiently, then feed the resulting feature maps into Transformer blocks for global reasoning. This combines the efficiency and inductive bias of CNNs with the global modeling of Transformers.

G23. What is the difference between a Transformer encoder and decoder? A: The encoder processes the entire input sequence simultaneously using self-attention. The decoder generates output tokens autoregressively (one at a time), using masked self-attention (to prevent looking ahead) and cross-attention to the encoder output.

G24. What is the masked self-attention in the Transformer decoder, and why is it necessary? A: Masked self-attention prevents the decoder from attending to future positions in the output sequence by setting attention scores for future tokens to negative infinity (before softmax). This enforces the autoregressive property: each token can only depend on previously generated tokens.

G25. Why is the quadratic complexity of self-attention a bigger problem for video than for images? A: A video has an additional time dimension. Treating video frames as tokens or using 3D patches dramatically increases N (number of tokens). For example, a 10-frame clip with 16x16 patches per frame has N = 10 * (224/16)^2 = 1960 tokens, and the attention matrix is 1960x1960, much larger than for a single image.

G26. What is a Transformer 'token' in the context of NLP vs computer vision? A: In NLP, a token is typically a word or subword unit represented as an embedding vector. In computer vision, a token is usually a flattened image patch (e.g., 16x16 pixels) projected to an embedding vector. Both are fed into the same self-attention mechanism.

G27. How does the Vision Transformer handle images of different aspect ratios? A: Standard ViT requires fixed-size square inputs (resized and cropped). Variants like NaViT (Native Resolution ViT) process images at their native resolution by packing multiple images into a single sequence, reducing wasted computation on padding.

G28. What is the difference between absolute and relative position embeddings? A: Absolute embeddings encode the exact position of each token (e.g., position 5 gets embedding e_5). Relative embeddings encode the distance between tokens (e.g., token at i attending to token at j uses embedding of distance i-j). Relative embeddings generalize better to longer sequences.

G29. Why do some vision Transformers use hierarchical architectures (e.g., Swin Transformer)? A: Hierarchical Transformers like Swin use windowed self-attention within local windows and shifted windows across layers. This reduces complexity from O(N^2) to O(N) while maintaining multi-scale feature maps similar to CNN pyramids, making them more efficient for dense prediction tasks.

G30. True or False: Self-attention in Transformers is inherently permutation equivariant. A: True. If you permute the input tokens, the output tokens are permuted in the same way (each output corresponds to the same input token, just in a new position). This is why position embeddings are essential to break symmetry and encode spatial/sequential information.

G31. Explain the role of position embeddings in Vision Transformers. What would happen without them? A: Self-attention is permutation-invariant — it treats input tokens as an unordered set. Without position

embeddings, shuffling image patches would produce the exact same output. Position embeddings inject spatial information by adding a learned or fixed vector to each patch token based on its position. Without them, the model could not distinguish a patch in the top-left from a patch in the bottom-right, making spatial reasoning impossible.

G32. What is the difference between self-attention and cross-attention? A: Self-attention computes attention weights where Queries, Keys, and Values all come from the SAME input sequence (Q=K=V=X). Cross-attention computes attention where Queries come from one sequence and Keys/Values come from ANOTHER sequence. Self-attention is used in encoder and decoder for intra-sequence relationships. Cross-attention is used in decoder-to-encoder connections (e.g., DETR queries attending to image features, or text queries attending to image patches in multi-modal models).

G33. Why does ViT need much more data than CNNs to achieve competitive performance? A: CNNs have strong inductive biases built-in: locality (nearby pixels matter more), translation invariance (same filter slides everywhere), and weight sharing. These are hard-coded architectural priors that work well even with limited data. Transformers have NO built-in inductive bias for images — they must learn spatial relationships, locality, and feature hierarchies entirely from data. Without massive pre-training datasets (ImageNet-21k, JFT), ViT underperforms CNNs.

G34. What is the computational complexity of self-attention, and why is it a bottleneck for high-resolution images? A: Self-attention computes Q·K^T, which is an N×N matrix where N = number of tokens. For an H×W image with P×P patches, N = (H·W)/P². Complexity is O(N²·D). For a 1024×1024 image with 16×16 patches, N = 4096 → attention matrix is 4096² ≈ 16M entries. This is much heavier than CNNs (O(N) with weight sharing). Solutions: sparse attention, windowed attention (Swin), linear attention, or hybrid CNN-Transformer architectures.

SECTION G: ATTENTION AND VISION TRANSFORMERS (34 Questions)

SECTION H: MIXED / INTEGRATED / DESIGN QUESTIONS (20 Questions)

H1. A student applies histogram equalization directly to each RGB channel of a sunset photograph. Describe the visual result and the correct procedure. A: The result will have unnatural, posterized colors. Equalizing each channel independently applies different non-linear remappings to R, G, B, destroying their ratios and thus hue. Correct procedure: Convert to LAB, apply HE/CLAHE to L channel only, convert back to RGB. This preserves hue and saturation while enhancing contrast.

H2. Design a complete pipeline to automatically detect and count LEGO bricks using traditional computer vision only. List each step. A: 1) Preprocessing: convert to LAB, equalize L channel. 2) Color segmentation: threshold in HSV or LAB to isolate LEGO colors. 3) Morphological operations: opening to remove noise, closing to fill gaps. 4) Connected components: label each region. 5) Filtering: remove small/large components by area. 6) Feature extraction: area, perimeter, circularity, aspect ratio. 7) Classification: use shape features to distinguish brick types. 8) Counting: number of components = number of bricks.

H3. Design a complete pipeline using deep learning to detect and count LEGO bricks. Compare with the traditional approach. A: 1) Data: collect and annotate images with bounding boxes. 2) Model: use YOLOv8 or Faster R-CNN. 3) Training: pre-train on COCO, fine-tune on LEGO dataset with augmentation. 4) Inference: run detector, apply NMS. 5) Counting: number of boxes after NMS. Comparison: Traditional requires hand-crafted rules and is fragile to changes. DL learns features automatically, robust to variations, generalizes to new backgrounds, easily extended to new classes.

H4. A CNN classifier achieves 99% training accuracy but only 65% validation accuracy. Diagnose the problem and propose three solutions. A: This is severe overfitting. The model memorized training data. Solutions: 1) Add Dropout (especially in FC layers). 2) Data augmentation: random crops, flips, rotations, color jitter. 3) Early stopping when validation loss plateaus. 4) Reduce model capacity. 5) L2 weight decay. 6) Transfer learning from pre-trained model.

H5. Explain the relationship between receptive field, kernel size, and network depth. If you want a 15x15 RF using only 3x3 convolutions with stride 1, how many layers? A: The receptive field grows by (K-1) per layer with stride 1. For K=3, each layer adds 2 to RF. Starting from 1: 1 + 2n = 15 -> n = 7 layers.

H6. Consider transformation matrix T = [[0,2,1],[2,0,1],[0,0,1]]. Is this affine or projective? What specific operations? What is T(1,2)? A: Affine (last row [0,0,1]). x' = 2y+1, y' = 2x+1. Operations: swap x and y, scale by 2, translate by (1,1). T(1,2) = (5,3).

H7. A microscopy image has touching blood cells. Basic thresholding gives a single blob for three touching cells. Explain step-by-step how to get individual cell segmentations. A: 1) Apply adaptive thresholding or trained U-Net for binary mask. 2) Compute distance transform of binary mask. 3) Find local maxima (cell centers). 4) Apply Watershed using maxima as markers. Watershed treats distance transform as topographic map and floods from markers, creating boundaries at ridges where cells touch. 5) Alternatively: use Mask R-CNN for end-to-end instance segmentation.

H8. Why is it generally better to use a Gaussian filter before downsampling an image? A: Downsampling without pre-filtering causes aliasing (high frequencies fold into low frequencies, creating moire patterns). A Gaussian filter acts as a low-pass filter, removing frequencies above the Nyquist limit of the downsampled resolution, preventing aliasing.

H9. A SIFT descriptor is 128-dimensional. If you have 1000 keypoints in each of two images, how many distance computations are needed for brute-force matching? A: Brute-force compares every descriptor in image A to every descriptor in image B: 1000 * 1000 = 1,000,000 distance computations. This is why FLANN is preferred for large-scale matching.

H10. Why does the VGG architecture use only 3x3 convolutions despite needing many layers? A: A stack of three 3x3 convolutions has the same receptive field as one 7x7 (7x7) but with fewer parameters (27C^2 vs 49C^2) and more non-linearities (3 ReLUs vs 1). The uniformity also simplifies design and analysis.

H11. What is the difference between a semantic segmentation network and an autoencoder? A: Both have encoder-decoder structures. However, autoencoders reconstruct the input (unsupervised) and their bottleneck learns compressed representations. Semantic segmentation networks produce class labels per pixel (supervised) and their decoder recovers spatial resolution for pixel-wise classification, often with skip connections for detail.

H12. You need to build a real-time face detection system for a mobile app. Compare Haar cascades, MTCNN, and MobileNet-SSD for this task. A: Haar cascades: very fast but outdated, poor accuracy on challenging poses/lighting. MTCNN: good accuracy, detects faces + landmarks, but slower and heavier. MobileNet-SSD: best balance - lightweight depthwise separable convolutions, fast enough for mobile, good accuracy. Recommended: MobileNet-SSD or newer lightweight detectors (YOLOv8-nano).

H13. What is the difference between training a GAN and training a standard classifier in terms of loss curves? A: In a classifier, loss steadily decreases and accuracy increases. In a GAN, the discriminator and generator losses oscillate adversarially. There is no single ‘convergence’ metric - D and G are in a game. Stable training shows D loss near log(2) and generated sample quality improving over time, but losses alone don’t indicate success.

H14. How would you handle a dataset where 95% of images are class ‘normal’ and 5% are class ‘defective’ for a quality inspection CNN? A: 1) Use class-weighted loss to penalize misclassifying minority class more. 2) Oversample defective images or undersample normal images. 3) Use data augmentation specifically on defective class. 4) Use appropriate metrics (F1, precision/recall) instead of accuracy. 5) Consider anomaly detection or GAN-based synthetic data generation for defective class.

H15. Explain why a 1x1 convolution with 64 filters applied to a 128-channel feature map is not just scaling each channel by a constant. A: A 1x1 convolution computes a linear combination across all input channels at each spatial location. It acts as a fully-connected layer per pixel across the channel dimension. With 64 output filters, each output channel is a different weighted sum of all 128 input channels, enabling channel-wise dimensionality reduction, expansion, and cross-channel information mixing.

H16. A homography perfectly aligns two images of a painting on a wall. What does this tell you about the geometry of the scene and camera motion? A: The painting is approximately planar. A homography perfectly maps between two views of a planar surface. If the camera only rotated about its center (no translation), a homography would also relate the images regardless of scene geometry. Since alignment is perfect, either the painting is planar or the camera underwent pure rotation.

H17. What would be the consequence of using a very large block size (e.g., 101x101) in adaptive thresholding? A: A very large block size makes the threshold nearly global, losing the benefit of local adaptation. It may fail to handle local illumination variations. Conversely, a very small block size (e.g., 3x3) makes the threshold overly sensitive to local noise. A moderate size (e.g., 11x11 to 31x31) is typically optimal.

H18. In a Vision Transformer, why might early layers show more uniform attention distributions while deeper layers show more focused attention? A: Early layers have not yet learned semantic concepts, so attention is more diffuse as the model explores relationships. Deeper layers have learned to recognize objects and parts, so attention becomes more focused on semantically relevant regions (e.g., attending from a face patch to body patches of the same person).

H19. Compare the parameter efficiency of ResNet-50 (~25.6M params) and VGG-16 (~138M params) despite ResNet being deeper. A: ResNet uses bottleneck blocks (1x1 -> 3x3 -> 1x1) that dramatically reduce channel dimensions before expensive convolutions. VGG uses stacks of 3x3 convolutions at full channel depth without bottlenecks. Additionally, ResNet uses global average pooling instead of large fully connected layers, saving millions of parameters.

H20. You have two cameras looking at the same scene from different viewpoints. The scene contains both a flat wall and a statue. Can a single homography align both images? Why or why not? A: No. A single homography can align the flat wall (planar surface) perfectly, but the statue is non-planar (3D object). Points on the statue at different depths undergo different projective transformations when the camera moves. Only epipolar geometry (fundamental/essential matrix) can describe the relationship for the entire general 3D scene with camera translation.

SECTION I: TRUE / FALSE (WITH JUSTIFICATION)

I1. T/F: A 3x3 Gaussian filter and a 3x3 mean filter always produce visually similar results. A: False. The Gaussian gives higher weight to the center pixel, producing smoother, more natural blur. The mean filter treats all pixels equally, creating slightly blockier results and more pronounced artifacts at edges.

I2. T/F: The Harris corner detector is invariant to rotation. A: True. The structure tensor M uses gradients in all directions. Rotating the image rotates the eigenvectors but leaves the eigenvalues unchanged. Since corner detection depends on eigenvalues (not orientation), it is rotation-invariant.

I3. T/F: SIFT descriptors are invariant to affine transformations. A: False. SIFT is invariant to scale and rotation. It is NOT fully invariant to affine transformations (e.g., strong perspective foreshortening). ASIFT (Affine-SIFT) was specifically designed for affine invariance.

I4. T/F: Max pooling layers have trainable parameters. A: False. Max pooling performs a fixed operation (taking the maximum). There are no learnable weights or biases. Average pooling also has no parameters.

I5. T/F: You should always use dropout during model evaluation/inference. A: False. Dropout is a training-only regularization technique. During inference, all neurons should be active to use the full model capacity. Applying dropout at test time would produce random, inconsistent predictions.

I6. T/F: Replacing max pooling with strided convolution always increases the number of trainable parameters. A: True. Max pooling has zero parameters. Strided convolution has (K^2*C_in+1)*C_out parameters. Therefore, replacing pooling with strided convolutions adds trainable parameters.

I7. T/F: In a GAN, the generator and discriminator are trained simultaneously with the same learning rate. A: False (usually). While they are trained alternately, they often require different learning rates or update frequencies. The discriminator is frequently trained more steps per generator step to maintain a healthy training balance. Using the same rate for both can lead to one dominating the other.

I8. T/F: A Vision Transformer can process images of varying sizes without any modification. A: False (mostly). While the Transformer itself is flexible, the patch embedding and position embeddings are typically fixed to a specific number of patches. To handle varying sizes, you would need to adjust the position embeddings or use an adaptive approach. Standard ViT implementations use fixed input sizes.

I9. T/F: Optical flow always provides the true physical motion of objects in a scene. A: False. Optical flow measures apparent motion of brightness patterns. It fails when: brightness changes without motion (shadows), there is no texture (uniform surfaces), the aperture problem causes ambiguity, or occlusions occur.

I10. T/F: U-Net can be used for image generation tasks, not just segmentation. A: True. The U-Net encoder-decoder architecture with skip connections is essentially an image-to-image mapping network. It is used in diffusion models, VAEs, and super-resolution as the backbone generator because it preserves spatial detail while transforming one image into another.

I11. T/F: A homography can perfectly align two images of a general 3D scene taken from different positions. A: False. A homography perfectly maps between two views of a planar surface or when the camera rotates about its center (no translation). For a general 3D scene with camera translation, points at different depths move differently (parallax), so no single homography can align the entire scene.

I12. T/F: Batch normalization and dropout serve the same purpose and should not be used together. A: False. While both help regularization, they work differently. Batch norm stabilizes training and reduces internal covariate shift. Dropout prevents co-adaptation. They CAN be used together (and often are), though batch norm sometimes reduces the need for dropout.

I13. T/F: A single convolutional layer with 64 filters always has fewer parameters than a fully connected layer mapping the same input to 64 outputs. A: True (for reasonably sized inputs). A conv layer shares weights spatially: params = (K^2*C_in+1)*C_out. An FC layer has unique weights per input-output pair: params = (H*W*C_in)*C_out + C_out. For typical images, the FC layer has vastly more parameters.

I14. T/F: GrabCut requires the user to label every pixel in the image. A: False. GrabCut is semi-automatic. The user only needs to provide a rough rectangle or mask. The algorithm iteratively refines the segmentation using GMMs and graph cuts automatically.

I15. T/F: Vision Transformers inherently understand spatial relationships without any additional mechanism. A: False. Self-attention is permutation-invariant. Without position embeddings (learned or sinusoidal), a ViT would be completely blind to spatial arrangement. Position embeddings are essential.

End of Complete Exam Preparation Guide. Every slide covered. Good luck!

SECTION A: IMAGES AND FILTERING (42 Questions)

A1. What is the effect of adding the same constant to all three RGB channels (assuming no clipping)? A: It increases brightness/luminance uniformly while preserving hue and saturation, because the ratios between R, G, and B remain unchanged.

A2. Why does OpenCV load color images as BGR by default, and what problem does this cause? A: OpenCV uses BGR for historical reasons. Displaying with matplotlib (expects RGB) without conversion makes colors appear swapped (e.g., sky looks reddish). Use cv2.cvtColor(img, cv2.COLOR_BGR2RGB).

A3. What is the expected result of histogram equalization on a grayscale image? A: It spreads the intensity distribution uniformly across [0,255], maximizing global contrast. Dark images become brighter; low-contrast images gain visible detail.

A4. Can you predict the effect of equalizing R, G, and B channels independently? A: No, and it is not recommended. Independent equalization applies different non-linear remappings to each channel, destroying the relative ratios that define hue. Result: unnatural, posterized colors.

A5. What is the correct procedure for histogram equalization on a color image? A: Convert BGR->LAB, apply HE or CLAHE to the L (lightness) channel only, then convert back to BGR/RGB. This preserves hue and saturation while enhancing contrast.

A6. Why are Gaussian filters generally preferable to simple mean filters? A: 1) Natural weighting (closer pixels matter more). 2) No sharp frequency cutoff -> no ringing. 3) Separable -> much faster for large kernels. 4) Scale-space property. 5) Closer to physical blur models.

A7. What is the separability property of the Gaussian filter, and why does it matter computationally? A: A 2D Gaussian can be decomposed into two 1D Gaussians: G(x,y) = G(x)*G(y). A kxk Gaussian blur becomes two kx1 passes, dropping complexity from O(k^2*N) to O(2k*N).

A8. A 5x5 Gaussian filter is applied to a 100x100 image. How many multiplications per pixel are saved by separability? A: Direct: 25 multiplications/pixel. Separable: 5+5 = 10 multiplications/pixel. Savings: 15/25 = 60% reduction.

A9. Why is the median filter particularly effective for salt and pepper noise? A: Salt & pepper noise sets random pixels to 0 or 255 (the extremes). The median of a neighborhood selects the middle value, which is almost never an extreme outlier. Thus outliers are completely ignored.

A10. Compare edge preservation among mean, Gaussian, median, and bilateral filters. A: Mean and Gaussian blur across edges. Median preserves sharp edges (selects existing pixel values). Bilateral preserves edges because intensity similarity weight drops to near zero across edges.

A11. What makes the bilateral filter 'edge-preserving'? A: Its weight depends on both spatial distance AND intensity difference. At an edge, pixels on the other side have very different intensities, so their weights become negligible, preventing blurring across the edge.

A12. A grayscale image has a median intensity of 220 (0-255 scale). After histogram equalization, will it become lighter or darker? A: Darker. A median of 220 means >50% of pixels are very bright. HE spreads the histogram uniformly, so many bright pixels must be remapped to lower values to flatten the distribution.

A13. If an image already has a uniform histogram, what is the effect of histogram equalization? A: Minimal to no visible change. The CDF is already approximately linear, so the mapping T(v) is nearly the identity function.

A14. Why is CLAHE generally preferred over standard HE for natural photographs? A: Global HE can over-amplify noise in nearly uniform regions and create unnatural contrast. CLAHE operates locally on tiles and clips histogram peaks before equalization, producing more natural results.

A15. Describe a scene and camera setup that would produce two very different optical flow fields. A: Setup 1: Static camera, moving scene (waves on beach) -> flow shows wave motion, background zero. Setup 2: Moving camera, static scene (car driving through city) -> flow shows focus of expansion with radiating outward vectors.

A16. What filter corresponds to the kernel [[1,2,1],[2,4,2],[1,2,1]]? A: Un-normalized 3x3 Gaussian kernel (approximation). When normalized by dividing by 16, it becomes a standard Gaussian smoothing filter / low-pass filter.

A17. What operation does the kernel [[0,-1,0],[-1,4,-1],[0,-1,0]] perform? A: Laplacian operator (discrete second derivative). It highlights regions of rapid intensity change (edges). Sum is 0, so it responds to changes, not uniform regions.

A18. What are the Sobel operator kernels for Gx and Gy? A: Gx = [[-1,0,1],[-2,0,2],[-1,0,1]]; Gy = [[-1,-2,-1],[0,0,0],[1,2,1]] (transpose of Gx).

A19. How do you compute gradient magnitude and direction from Sobel outputs? A: Magnitude = sqrt(Gx^2 + Gy^2). Direction = atan2(Gy, Gx).

A20. Explain the difference between 'valid', 'same', and 'full' convolution modes. A: Valid: no padding, output smaller. Same: padding so output equals input size (stride 1). Full: maximum padding, output larger than input.

A21. What is the kernel size rule-of-thumb for a Gaussian filter with standard deviation sigma? A: k = 6*sigma + 1. This captures approximately 99.7% of the Gaussian distribution (3 sigma on each side).

A22. What is the scale-space property of Gaussian filtering? A: Applying two Gaussian filters with sigma1 and sigma2 sequentially is equivalent to applying one Gaussian with sigma = sqrt(sigma1^2 + sigma2^2).

A23. A 3x3 mean filter is applied twice. Is the result equivalent to a single larger filter? A: Yes, it produces a 5x5 filter with weights following a triangular (binomial) distribution. Same receptive field as 5x5, but not exactly a 5x5 mean filter.

A24. What are the main noise models in digital images, and which filter is best for each? A: Gaussian noise -> Gaussian or mean filter. Salt & pepper -> median filter. Speckle -> adaptive filters or homomorphic filtering. Poisson -> variance-stabilizing transforms or specialized denoising.

A25. When does histogram equalization hurt rather than help? A: When the image already has good contrast (may over-amplify noise). When certain intensities should be preserved (medical images with specific tissue appearances). When the histogram is not compressible.

A26. What is the Brightness Constancy Equation in optical flow? A: Ixu + Iyv + It = 0, where (u,v) is the flow vector, Ix and Iy are spatial gradients, and It is the temporal derivative.

A27. Why might optical flow estimate zero motion for a rotating uniform white sphere? A: Because the surface is uniform and lighting is static, there are no brightness changes between frames. The brightness constancy assumption yields zero flow despite physical rotation.

A28. What is the aperture problem in optical flow? A: When viewing through a small window on an edge, only the motion component perpendicular to the edge is observable. The tangential component is ambiguous.

A29. A medical X-ray is dark with histogram compressed in 0-80 range. Will HE help? What about CLAHE? A: Both will help by stretching the range. CLAHE is preferable because global HE might over-amplify noise in dark regions. CLAHE's clipLimit prevents this.

A30. Compute the output of a 3x3 mean filter on patch [[10,20,30],[40,50,60],[70,80,90]]. A: Sum = 450. Mean = 50. The center pixel becomes 50. In valid convolution on a larger image, every output pixel is the mean of its 3x3 neighborhood.

A31. True or False: Adding the same value to all RGB channels changes the hue of the image. A: False. The ratios between R, G, and B remain the same, so hue is preserved. Only brightness increases.

A32. True or False: A Gaussian filter and a mean filter of the same size always produce visually similar results. A: False. Gaussian gives higher weight to the center, producing smoother, more natural blur. Mean treats all pixels equally, creating blockier results.

A33. Why is HSV useful for color-based segmentation? A: Because lighting changes mostly affect the Value channel, leaving Hue relatively stable. Thresholding by Hue isolates specific colors regardless of brightness variations.

A34. What does it mean that LAB is 'perceptually uniform'? A: Equal Euclidean distance in LAB space corresponds approximately to equal perceived color difference by the human visual system.

A35. For a 224x224 RGB image, what is the receptive field after three 3x3 convolutions with stride 1 and no padding? A: Each 3x3 layer adds 2 to the receptive field. Starting from 1x1: after 3 layers, RF = 1 + 3*2 = 7x7.

A36. What is the purpose of non-maximum suppression in the Canny edge detector? A: To thin the edges to a single pixel width. After computing gradient magnitude and direction, NMS keeps only pixels that are local maxima along the gradient direction, suppressing weaker neighboring pixels. This produces sharp, continuous edges instead of thick, blurred edge bands.

A37. Why does Canny edge detection apply Gaussian smoothing before computing gradients? A: Because gradient operators (Sobel) are sensitive to noise. The second derivative (Laplacian) is especially noise-amplifying. Gaussian smoothing reduces high-frequency noise while preserving edge structure, preventing the detector from producing spurious edges on noise.

A38. A 3x3 convolution kernel produces a vertical line response. What are its values? A: A vertical edge detector highlights vertical intensity changes. One standard form is `[[-1,0,1],[-1,0,1],[-1,0,1]]` (Sobel-like vertical). Or simpler: `[[0,1,0],[0,1,0],[0,1,0]]` detects vertical bright lines on dark background. The key insight: positive values in center column, zero or negative in side columns → responds to vertical structures.

A39. An image has median brightness 200 (out of 255). After histogram equalization, would it become brighter or darker? Explain. A: Darker. A median of 200 means the image is already bright (values clustered in upper range). HE spreads the histogram uniformly across the full [0,255] range. Since many pixels are already near the top, they must move down (darker) to make room for the uniform distribution. The cumulative distribution is flattened, which pulls bright pixels toward the mean.

A40. What is the purpose of non-maximum suppression in the Canny edge detector? Explain with the double thresholding step. A: NMS thins edges to single-pixel width by keeping only local maxima along the gradient direction. Without NMS, edges would be thick bands. Double thresholding then classifies: strong edges (>high_threshold) are kept, weak edges (between thresholds) are kept only if connected to strong edges via hysteresis. This produces continuous, clean contours while suppressing noise.

A41. Why is bilinear interpolation preferred over nearest neighbor for image warping? A: Bilinear computes a weighted average of 4 neighboring pixels, producing smooth, continuous values. Nearest neighbor simply rounds to the closest pixel, creating jagged edges, blocky artifacts, and aliasing (staircase effects on diagonals). Bilinear is the standard for `warpAffine`/`warpPerspective` because it balances quality and speed.

A42. What happens if you downsample an image without anti-aliasing (low-pass filtering first)? A: Aliasing occurs. High-frequency content (fine details, sharp edges, textures) that exceeds the Nyquist rate of the new resolution folds into lower frequencies, creating moiré patterns, jagged edges, and false structures. Correct approach: apply Gaussian blur first to remove frequencies that cannot be represented at the new resolution, then downsample.

SECTION B: FEATURES, LOCAL FEATURES, AND WARPING (44 Questions)

B1. How can we change the sensitivity of the Harris corner detector to detect more or fewer corners? A: Decrease k -> more corners. Lower threshold -> more corners. Reduce minDistance -> more corners (but duplicates). Decrease blockSize -> finer corners. For fewer corners, do the opposite.

B2. What do the eigenvalues of the Harris structure tensor tell us about local image structure? A: Both small -> flat region. One large, one small -> edge. Both large -> corner. Both large and similar -> isotropic corner (corner with similar gradients in all directions).

B3. Match the following eigenvalue pairs to regions: (0,0), (4,4), (0.1,5). A: (0,0) -> flat region. (4,4) -> corner. (0.1,5) -> edge.

B4. Why is Shi-Tomasi considered more stable than Harris? A: Shi-Tomasi uses min(lambda1, lambda2) directly. Harris combines eigenvalues through the response formula R = det(M) - k*trace(M)^2, which can be sensitive to the choice of k.

B5. Describe the FAST corner detector algorithm. A: Tests 16 pixels on a Bresenham circle of radius 3 around pixel p. If N contiguous pixels are all brighter than I(p)+t or all darker than I(p)-t, p is a corner. Non-maximum suppression removes clustered responses. Very fast.

B6. Do SIFT and ORB operate in color or grayscale? Explain. A: Grayscale. Keypoint detection relies on intensity gradients and local contrast, not color. Running on R, G, B independently would produce redundant detections of the same geometric structures.

B7. What are the main steps of the SIFT algorithm? A: 1) Scale-space extrema detection using DoG pyramid. 2) Keypoint localization and refinement. 3) Orientation assignment from local gradient histogram. 4) Descriptor computation: 4x4 spatial bins x 8 orientations = 128D vector.

B8. What is SIFT invariant and NOT invariant to? A: Invariant to: scale, rotation, illumination changes. NOT invariant to: affine transformations, strong perspective foreshortening.

B9. How does ORB achieve rotation invariance? A: ORB measures the intensity centroid of the patch and rotates the BRIEF descriptor according to the dominant orientation. This is called ‘steered BRIEF’.

B10. Compare SIFT and ORB in terms of descriptor type and matching speed. A: SIFT: 128D float descriptor, L2 distance, slower but more robust. ORB: 256-bit binary descriptor, Hamming distance (XOR+popcount), much faster but less robust to scale.

B11. What is Lowe’s ratio test, and why is it important? A: For each descriptor, find 2 nearest neighbors. Accept match only if d(best)/d(second) < 0.7. This rejects ambiguous matches where the two best candidates are too similar, often indicating repetitive patterns or non-distinctive descriptors.

B12. When would you use FLANN instead of Brute Force matching? A: FLANN is preferred for large datasets because it uses approximate nearest neighbor search (KD-trees, k-means trees) and is much faster. Brute Force is better for small datasets where exact matching is needed.

B13. Explain the role of RANSAC in homography estimation. A: Even after Lowe’s ratio test, feature matching produces outliers. RANSAC randomly samples 4 correspondences, computes a candidate homography H, counts inliers, and repeats. It keeps the H with most inliers and re-estimates using all inliers. Robust to up to 50% outliers.

B14. How many iterations does RANSAC need? Give the formula. A: N = log(1-p) / log(1-w^s), where p = desired success probability (e.g., 0.99), w = inlier fraction, s = sample size (4 for homography).

B15. A planar object is photographed from two different angles. What transformation relates the two images? What if the object is non-planar? A: Planar object: homography (8 DOF). Non-planar object with camera translation: epipolar geometry (fundamental/essential matrix), not just homography. If camera only rotates about its center: homography regardless of scene.

B16. What is the difference between affine and perspective transformation? Give two real-world examples. A: Affine (6 DOF): preserves parallel lines. Examples: rotating/scaling a logo on a flat document; shear in scanned documents. Perspective (8 DOF): does NOT preserve parallel lines (they converge). Examples: perspective correction of a building photo; stitching two panorama photos.

B17. How many point correspondences are needed to estimate a homography, and why? A: 4 point pairs. A homography has 8 DOF (3x3 matrix up to scale). Each point correspondence gives 2 equations. 4 points x 2 equations = 8 equations for 8 unknowns. No 3 points can be collinear.

B18. In image warping, why is backward warping preferred over forward warping? A: Backward warping maps each destination pixel to a source location via inverse transform and samples. It guarantees every destination pixel gets a value (no gaps). Forward warping maps source to destination, leaving holes when compression occurs and requiring overlap handling when expansion occurs.

B19. What OpenCV functions perform affine and perspective warping? A: Affine: cv2.warpAffine(image, M, (w,h)) with 2x3 matrix M. Perspective: cv2.warpPerspective(image, H, (w,h)) with 3x3 matrix H. Both use backward warping with bilinear interpolation by default.

B20. Given transformation matrix T = [[0,2,1],[2,0,1],[0,0,1]], what type is it and what is T(1,2)? A: Affine (last row is [0,0,1]). It swaps x and y, scales by 2, and translates by (1,1). T(1,2) = (2*2+1*, *2*1+1) = (5,3).

B21. What is the aperture problem, and how does the Harris detector overcome it? A: The aperture problem is the ambiguity of motion along an edge when viewed through a small window. Harris overcomes it by analyzing the structure tensor: on an edge, one eigenvalue is large and one is small ($R < 0$), so edges are rejected. Corners have both eigenvalues large ($R > 0$), where 2D motion is unambiguous.

B22. You run Harris on a featureless white wall. What do you expect? A: No corners detected. The structure tensor M has both eigenvalues near zero everywhere, so $R \sim 0$, which is below any reasonable threshold.

B23. You need to find correspondences between two photos of the same building taken 5 years apart. The building was repainted. Which detector? A: SIFT is preferred because its descriptor is based on gradient orientations, which are robust to color and illumination changes. ORB is more intensity-based and sensitive to appearance changes. Always apply Lowe's ratio test and RANSAC.

B24. What is the Difference-of-Gaussians (DoG) pyramid used for in SIFT? A: It approximates the Laplacian of Gaussian (LoG) and is used to detect scale-space extrema. Local maxima/minima across scales indicate blob-like structures at different sizes.

B25. True or False: The Harris corner detector is invariant to rotation. A: True. Rotating the image rotates the eigenvectors of the structure tensor but leaves the eigenvalues unchanged. Since corner detection depends on eigenvalues (not orientation), it is rotation-invariant.

B26. True or False: SIFT descriptors are invariant to affine transformations. A: False. SIFT is invariant to scale and rotation, but NOT to affine transformations. ASIFT (Affine-SIFT) was designed specifically for affine invariance.

B27. What are the ideal properties of a local image feature? A: Repeatability (same scene -> same features), distinctiveness (easy to tell apart), locality (robust to occlusion), quantity (enough to cover scene), accuracy (precise localization), efficiency (fast to compute and match).

B28. Why are corners better than edges for matching? A: Corners are locally unique (two strong gradients in different directions). Edges are ambiguous because you can slide along an edge without changing appearance. Flat regions have no information.

B29. Explain non-maximum suppression in the context of corner detection. A: After computing corner response R across the image, NMS keeps only local maxima. In a neighborhood, only the pixel with the highest R is retained as a corner; others are suppressed. This removes clustered, duplicate corners.

B30. What is BRIEF, and how does ORB improve it? A: BRIEF is a binary descriptor where each bit is the result of an intensity comparison between two pixel locations around the keypoint. ORB adds orientation to FAST keypoints and steers/rotates the BRIEF pattern to achieve rotation invariance.

B31. What is the Hamming distance, and why is it used for ORB matching? A: Hamming distance counts the number of differing bits between two binary strings. It can be computed extremely fast using XOR and popcount (CPU instructions), making ORB matching much faster than SIFT's L2 distance.

B32. When should you use homography vs affine for image alignment? A: Use homography when: same plane viewed from different angles, image stitching, perspective correction, AR on planar surfaces. Use affine when: simple 2D manipulations (rotation, scale, shear), the scene is far enough that perspective effects are negligible, or you need more stability with fewer parameters.

B33. What is the Direct Linear Transform (DLT) used for in RANSAC? A: DLT is the algebraic method to compute a homography (or other projective transformation) from a set of point correspondences. In RANSAC, DLT is applied to the minimal sample set (4 points) to generate a hypothesis H.

B34. How does the pyramid implementation of Lucas-Kanade help with larger motions? A: It builds a Gaussian pyramid of both images and computes optical flow from coarsest to finest level. At each level, the flow estimate from the coarser level is used to warp the finer level, effectively handling larger displacements that violate the small-motion assumption.

B35. What happens if you use SIFT matching without RANSAC to estimate a homography? A: Even with Lowe's ratio test, there are typically 10-40% outliers. Least-squares estimation on all matches (including outliers) would be heavily biased, producing a distorted and incorrect homography.

B36. Describe the difference between Euclidean, Similarity, Affine, and Projective transformations in terms of DOF and preserved properties. A: Euclidean: 3 DOF, preserves lengths/angles/areas. Similarity: 4 DOF, preserves angles and ratios of lengths. Affine: 6 DOF, preserves parallel lines and ratios of areas. Projective: 8 DOF, preserves only straight lines.

B37. What is the difference between a corner and a blob in feature detection? A: A corner is a point where two edges meet, detected by intensity changes in two directions (e.g., Harris). A blob is a region that stands out from its surroundings in terms of intensity or texture, detected at multiple scales (e.g., SIFT DoG extrema).

B38. Why does the Harris response R = det(M) - k*trace(M)^2 produce negative values on edges? A: On an edge, one eigenvalue is large (lambda1) and one is small (lambda2). det(M) = lambda1*lambda2 is small. trace(M)^2 = (lambda1+lambda2)^2 is dominated by lambda1^2. For large lambda1, k*trace^2 exceeds det(M), making R negative.

B39. What is the main advantage of ORB over SIFT for real-time applications? A: ORB is orders of magnitude faster due to: FAST detection (vs DoG), binary BRIEF descriptors (vs 128D float), and Hamming distance matching (XOR+popcount vs L2 norm). This makes it suitable for real-time AR, SLAM, and mobile applications.

B40. In OpenCV, how do you compute a homography from point correspondences with RANSAC? A: cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, ransacReprojThreshold). It returns the homography matrix H and a mask indicating inliers.

B41. For the following patches, which eigenvalue pair (λ_1, λ_2) of the Harris structure tensor corresponds to each? - Patch (a): strong horizontal edge - Patch (b): uniform gray region - Patch (c): corner with strong gradients in both directions A: Harris structure tensor eigenvalues indicate local structure: - (0, 0): flat/uniform region → corresponds to **patch (b)** (no gradient in any direction). - (4, 4): both eigenvalues large → corner → corresponds to **patch (c)** (strong gradients in both x and y). - (0.1, 5): one large, one small → edge → corresponds to **patch (a)** (strong gradient in one direction, almost none in perpendicular direction).

B42. A transformation matrix T maps (x,y,1) to (−2y+1, 2x+1, 1). What type of transformation is this, and what are the target coordinates of point (1,2)? A: The mapping `(x,y) → (−2y+1, 2x+1)` involves rotation (swapping x and y with sign change) and translation (+1, +1). Specifically, it is a similarity transformation: rotation by 90° (counterclockwise) combined with scaling by factor 2, plus translation. For point (1,2): x' = −2·2 + 1 = −3, y' = 2·1 + 1 = 3. Target coordinates: **(−3, 3)**.

B43. Do SIFT and ORB operate on color or grayscale images? Explain. A: Both operate on **grayscale** images. Keypoint detection relies on intensity gradients, not color information. Computing gradients on each color channel separately would produce redundant keypoints (one set per channel) that are not useful. The algorithms convert the image to grayscale first, then compute gradients and detect features based on intensity structure.

B44. You need to match correspondences between two image pairs: (1) a scene with texture but viewpoint change, and (2) a scene with repeated patterns (like a checkerboard). Which techniques would you use for each, and why? A: Pair 1 (texture + viewpoint): Use **SIFT** because it is invariant to scale and rotation, robust to viewpoint changes, and produces highly distinctive 128-D descriptors. Lowe's ratio test + RANSAC on homography handles outliers from viewpoint change.

Pair 2 (repeated patterns): Use **SIFT/ORB with geometric verification**. Repeated patterns cause many ambiguous matches (all squares look the same). After initial matching, **RANSAC** is essential to find the consistent geometric transformation. Without RANSAC, almost all matches would be wrong.
